

UNIwersytet Kardynała Stefana Wyszyńskiego
w Warszawie

Wydział Matematyczno-Przyrodniczy
Szkoła Nauk Ścisłych

Adam Waśko

Nr alb.: 115986

Kierunek studiów: informatyka

Analiza porównawcza i czynnikowa
wielokryterialnych algorytmów ewolucyjnych
w optymalizacji tras bezzałogowych
statków powietrznych
w misjach poszukiwawczo-ratowniczych

Praca magisterska

Promotor:

dr hab. inż. Krzysztof Trojanowski,
prof. UKSW

Warszawa, 2026

Streszczenie

Praca dotyczy porównania wielokryterialnych algorytmów ewolucyjnych w zadaniu optymalizacji tras zespołu bezzałogowych statków powietrznych (UAV) w misjach poszukiwawczo-ratowniczych (SAR) oraz ustalenia, które komponenty projektowe tych algorytmów odpowiadają za jakość uzyskiwanych rozwiązań. Problem sformułowano jako wariant marszrutyzacji pojazdów z dwoma sprzecznymi kryteriami, oboma podlegającymi minimalizacji: czasem zakończenia misji oraz współczynnikiem nieodkrytych ofiar (mierzącym opóźnienie w odnajdywaniu ofiar).

Badania przeprowadzono w czterech krokach. Najpierw wykazano, że hybrydowa inicjalizacja populacji, łącząca heurystyki konstrukcyjne z losowością, jest warunkiem koniecznym sensownej jakości – daje wyniki nieporównanie lepsze od inicjalizacji czysto losowej. Ponieważ punktem wyjścia było porównanie SMS-EMOA z NSGA-II, różniących się jednocześnie dwoma czynnikami, ich wpływ rozdzielono planem czynnikowym 2×2 na architekturę (steady-state vs. generacyjna) oraz kryterium selekcji (wkład do hypervolume vs. crowding distance). Następnie każdy z czterech wariantów dostrojono niezależnie narzędziem irace, aby ocenić wpływ strojenia i sprawdzić, czy porządek wariantów utrzymuje się po dostrojeniu. Na koniec przeprowadzono ablację operatorów mutacji na najlepszym dostrojonym wariantcie.

Eksperymenty na ośmiu instancjach testowych wykazały, że o jakości decyduje architektura algorytmu, podczas gdy kryterium selekcji nie ma istotnego wpływu; przewaga architektury generacyjnej jest najwyraźniejsza na instancjach trudnych i odporna na usunięcie dowolnej pojedynczej instancji. Strojenie poprawiło każdy wariant, lecz nie odwróciło tej hierarchii. Ablacja wykazała ponadto, że o jakości decyduje niewielki podzbiór operatorów mutacji: dwa z sześciu są niezbędne, a trzy wystarczają.

Słowa kluczowe

SMS-EMOA, NSGA-II, optymalizacja wielokryterialna, UAV, misje SAR, algorytmy ewolucyjne, front Pareto, hypervolume, irace, plan czynnikowy, ablacja operatorów mutacji

Title

Comparative and factorial analysis of multi-objective evolutionary algorithms for unmanned aerial vehicle path optimization in search and rescue missions

Abstract

This thesis concerns the comparison of multi-objective evolutionary algorithms for route optimization of a team of unmanned aerial vehicles (UAVs) in search and rescue (SAR) missions, and the identification of the design components responsible for solution quality. The problem is formulated as a variant of the vehicle routing problem with two conflicting objectives, both to be minimized: mission completion time and the fraction of undiscovered victims (a measure of the delay in locating victims). The study proceeds in four steps. First, hybrid population initialization – combining constructive heuristics with randomness – is shown to be a prerequisite for reasonable quality, yielding results incomparably better than purely random initialization. Since the starting point was a comparison of SMS-EMOA with NSGA-II, which differ in two design factors at once, their contributions were separated by a 2×2 factorial design over architecture (steady-state vs. generational) and selection criterion (hypervolume contribution vs. crowding distance). Each of the four variants was then tuned independently with the irace tool, to assess the effect of tuning and to check whether the ordering of the variants is preserved after tuning. Finally, an ablation of the mutation operators was performed on the best tuned variant.

Experiments on eight test instances showed that solution quality is determined by the algorithm architecture, whereas the selection criterion has no significant effect; the advantage of the generational architecture is most pronounced on hard instances and robust to the removal of any single instance. Tuning improved every variant but did not reverse this hierarchy. The ablation further showed that quality rests on a small subset of mutation operators: two of the six are essential, and three suffice.

Keywords

SMS-EMOA, NSGA-II, multi-objective optimization, UAV, SAR missions, evolutionary algorithms, Pareto front, hypervolume, irace, factorial design, ablation of mutation operators

Spis treści

Spis rysunków	6
Spis tabel	7
Spis algorytmów	8
Wykaz stosowanych oznaczeń	9
Wstęp	11
Rozdział 1. Problem i przegląd literatury	13
1.1. Misje poszukiwawczo-ratownicze i rola UAV	13
1.2. Pełne pokrycie i planowanie ścieżki UAV	14
1.3. Sformułowanie problemu	14
1.4. Klasyfikacja problemu	15
1.5. Optymalizacja wielokryterialna	16
1.6. Ewolucyjne algorytmy wielokryterialne	17
1.7. Powiązane prace i pozycjonowanie	19
Rozdział 2. Algorytmy i implementacja	21
2.1. Reprezentacja rozwiązania	21
2.2. Funkcje celu	22
2.2.1. Harmonogram lotu	22
2.2.2. Funkcja f_1 – czas zakończenia misji	24
2.2.3. Funkcja f_2 – współczynnik nieodkrytych ofiar	24
2.3. Inicjalizacja populacji	26
2.3.1. Perturbacja osobników heurystycznych	26
2.4. Operator krzyżowania	27
2.4.1. Adaptacja OX dla wielu UAV	27
2.5. Operatory mutacji	28
2.5.1. Procedura mutacji	29
2.6. Plan czynnikowy: cztery warianty algorytmu	29
2.7. SMS-EMOA	30
2.8. NSGA-II	31
2.9. SS-NSGA-II	32
2.10. GEN-SMS-EMOA	32
2.11. Podsumowanie wariantów	33
2.12. Szczegóły implementacyjne	33
Rozdział 3. Metodologia badań	34
3.1. Instancje testowe	34
3.2. Hypervolume jako miara jakości	34

3.3.	Narzędzia statystyczne	35
3.4.	Plan eksperymentów	37
3.4.1.	Eksperyment inicjalizacyjny	37
3.4.2.	Eksperyment czynnikowy	38
3.4.3.	Strojenie hiperparametrów	40
3.4.4.	Ablacja operatorów mutacji	41
Rozdział 4.	Wyniki	44
4.1.	Zasadność inicjalizacji hybrydowej	44
4.2.	Wpływ architektury i kryterium selekcji	45
4.3.	Strojenie hiperparametrów	48
4.3.1.	Dostrojone konfiguracje	48
4.3.2.	Który wariant wypadu najlepiej	48
4.4.	Ablacja operatorów mutacji	50
4.5.	Dyskusja	52
Rozdział 5.	Podsumowanie	54
5.1.	Realizacja celów pracy	54
5.2.	Główne wyniki	54
5.3.	Wkład pracy	56
5.4.	Ograniczenia i kierunki dalszych badań	56
Bibliografia		58
Dodatek A — opis kodu		61
	Struktura repozytorium	61
	Uruchamianie eksperymentów	61

Spis rysunków

2.1.	Schemat reprezentacji rozwiązania jako zestawu tras UAV	23
4.1.	Zbieżność HV : inicjalizacja hybrydowa a losowa (tcGB)	45
4.2.	Zbieżność HV czterech wariantów na tcW	47
4.3.	Front Pareto: SMS-EMOA domyślny vs dostrojony	49
4.4.	Ścieżka ablacji operatorów	51

Spis tabel

2.1.	Operatory mutacji zastosowane w algorytmie	28
2.2.	Cztery warianty algorytmu (plan czynnikowy 2×2)	33
3.1.	Charakterystyka ośmiu instancji testowych	35
3.2.	Parametry stałe wszystkich eksperymentów	37
3.3.	Plan czynnikowy 2×2 – cztery warianty algorytmu	39
4.1.	Inicjalizacja hybrydowa a losowa (SMS-EMOA)	44
4.2.	Mediany HV czterech wariantów (eksperyment czynnikowy)	46
4.3.	Wyniki trzech kontrastów czynnikowych	46
4.4.	Konfiguracje irace per wariant	48
4.5.	Ranking wariantów po strojeniu	49
4.6.	Przewaga architektury po strojeniu	50
4.7.	Ważność operatorów mutacji	52

Spis algorytmów

1	Order Crossover dla wielu UAV	28
2	Procedura mutacji	29
3	SMS-EMOA (steady-state, selekcja wg wkładu do HV)	31
4	NSGA-II (generacyjny, selekcja wg crowding distance)	32
5	Zachłanna eliminacja wsteczna operatorów mutacji	43

Wykaz stosowanych oznaczeń

Symbole

n	liczba regionów obszaru katastrofy
k	liczba UAV (dronów) w misji
x	rozwiązanie (zestaw tras wszystkich dronów)
π_j	trasa j -tego drona (sekwencja regionów)
f_1	czas zakończenia misji (makespan)
f_2	współczynnik nieodkrytych ofiar
D_j	czas zakończenia trasy j -tego drona
T_{cut}	okno oceny f_2 : czas zakończenia najszybszego drona ($\min_j D_j$)
$V(t)$	skumulowana liczba ofiar odkrytych do chwili t
V_{total}	całkowita liczba ofiar we wszystkich regionach
HV	hypervolume (miara jakości zbioru rozwiązań niezdominowanych)
$\overline{HV}$	zagregowane HV (średnia po instancjach z median po ziarnach)
$\mathbf{r}$	punkt referencyjny wskaźnika hypervolume
$\mathcal{A}$	zbiór rozwiązań niezdominowanych (argument HV)
M	liczba kryteriów optymalizacji ($M = 2$ w niniejszej pracy)
R	liczba niezależnych uruchomień na parę wariant–instancja ($R = 30$)
N	liczba instancji testowych w testach statystycznych
δ_i	różnica median HV na i -tej instancji (test Wilcoxona)
R_i	ranga wartości $ \delta_i $; R_1 – suma rang pierwszej grupy
W	statystyka testu Wilcoxona (W^+ , W^-)
A	wskaźnik Vargha-Delaneya (wielkość efektu)
m_1, m_2	liczności dwóch porównywanych grup we wskaźniku A
α	poziom istotności ($\alpha = 0,05$)
K	liczba testów w rodzinie (korekcja Holma-Bonferroniego)
μ	rozmiar populacji algorytmu ewolucyjnego
P	populacja robocza algorytmu
E	zewnętrzne archiwum rozwiązań niezdominowanych
L	ostatni (częściowy) front w procedurze selekcji; $ L $ – jego rozmiar
p_c, p_m	prawdopodobieństwo krzyżowania i mutacji
$\mathcal{O}$	zbiór operatorów mutacji ($\mathcal{O}_1, \dots, \mathcal{O}_6$)
$\mathcal{S}$	aktywny podzbiór operatorów (analiza ablacyjna)
w_i	waga i -tego operatora mutacji
N_{eval}	budżet ewaluacji funkcji celu

Skróty

UAV	Unmanned Aerial Vehicle (bezzałogowy statek powietrzny)
SAR	Search and Rescue (misja poszukiwawczo-ratownicza)
CPP	Coverage Path Planning (planowanie pełnego pokrycia)
VRP	Vehicle Routing Problem (problem marszrutyzacji pojazdów)
ENV	Expected Number of Victims (oczekiwana liczba ofiar)
EA	Evolutionary Algorithm (algorytm ewolucyjny)
MOEA	Multi-Objective Evolutionary Algorithm
SMS-EMOA	S-Metric Selection Evolutionary Multi-Objective Algorithm
NSGA-II	Non-dominated Sorting Genetic Algorithm II
SS-NSGA-II	Steady-State NSGA-II (wariant z planu czynnikowego)
GEN-SMS-EMOA	Generational SMS-EMOA (wariant z planu czynnikowego)
SPEA2	Strength Pareto Evolutionary Algorithm 2
OX	Order Crossover (krzyżowanie porządkowe)
HV	Hypervolume (wskaźnik jakości frontu Pareto)
IQR	Interquartile Range (rozstęp międzykwartyłowy)
irace	Iterated Racing for Automatic Algorithm Configuration

Wstęp

Misje poszukiwawczo-ratownicze prowadzone w obszarach dotkniętych katastrofami naturalnymi – trzęsieniami ziemi, powodzią czy lawinami – stawiają przed służbami ratunkowymi wyzwanie szybkiego przeszukania rozległego i często trudno dostępnego terenu. Bezzałogowe statki powietrzne (UAV) stały się w ostatnich latach cennym narzędziem wspierającym takie operacje, oferując dostęp do obszarów niedostępnych dla pojazdów lądowych oraz możliwość szybkiego przeszukiwania z powietrza. Skuteczność misji z wykorzystaniem zespołu dronów zależy jednak w dużej mierze od jakości zaplanowanych tras – zarówno pod względem czasu przeszukania całego obszaru, jak i kolejności odwiedzania regionów o różnym prawdopodobieństwie obecności ofiar.

Planowanie tras zespołu UAV dla misji poszukiwawczo-ratowniczych stanowi problem optymalizacji kombinatorycznej o charakterze wielokryterialnym. Dwa istotne kryteria – czas zakończenia misji oraz współczynnik nieodkrytych ofiar – pozostają ze sobą w konflikcie: szybsze zakończenie misji może wymagać pominięcia priorytetu regionów gęsto zaludnionych, a premiowanie wczesnego odkrywania ofiar może wydłużać całkowity czas operacji. W takich problemach nie istnieje pojedyncze rozwiązanie optymalne, lecz cały zbiór rozwiązań kompromisowych, z których żadne nie jest jednoznacznie lepsze od pozostałych (formalnie: front Pareto, Rozdział 1). Naturalnym narzędziem do ich rozwiązywania są ewolucyjne algorytmy wielokryterialne, których populacyjny charakter pozwala wyznaczać wiele kompromisów jednocześnie.

Punktem wyjścia niniejszej pracy był algorytm SMS-EMOA (*S-Metric Selection Evolutionary Multi-Objective Algorithm*), oceniający jakość całego zbioru rozwiązań jedną miarą i wykorzystujący ją bezpośrednio w mechanizmie selekcji (wskaźnik hypervolume, formalnie zdefiniowany w Rozdziale 3). Praca stanowi kontynuację badań nad optymalizacją tras UAV w misjach poszukiwawczo-ratowniczych (SAR, *Search and Rescue*) prowadzonych przez Trojanowskiego, Mikitiuka, Grzeszczaka i Guinanda [27, 28], w których zastosowano wcześniej przeszukiwanie lokalne oraz algorytm SPEA2 (*Strength Pareto Evolutionary Algorithm 2*). Podczas implementacji algorytmu SMS-EMOA zaobserwowano istotne podobieństwo strukturalne do algorytmu NSGA-II (*Non-dominated Sorting Genetic Algorithm II*) – oba opierają się na sortowaniu niezdominowanym i różnią się przede wszystkim kryterium selekcji środowiskowej oraz architekturą aktualizacji populacji. Spostrzeżenie to stało się bezpośrednią motywacją do przeprowadzenia systematycznego planu czynnikowego, rozkładającego oba algorytmy na dwa niezależne czynniki projektowe i badającego, który z nich faktycznie odpowiada za jakość uzyskiwanych wyników. Plan czynnikowy stanowi rdzeń pracy; uzupełniają go analizy inicjalizacji populacji, strojenia hiperparametrów oraz znaczenia poszczególnych operatorów mutacji.

Cel pracy. Głównym celem pracy jest porównanie wielokryterialnych algorytmów ewolucyjnych w optymalizacji tras bezzałogowych statków powietrznych dla misji poszukiwawczo-ratowniczych oraz ustalenie, które elementy ich konstrukcji – inicjalizacja populacji, architektura i kryterium selekcji, dobór hiperparametrów oraz zestaw operatorów mutacji – odpowiadają za jakość uzyskiwanych rozwiązań. Cel ten realizowany jest poprzez następujące cele szczegółowe:

- implementację czterech porównywanych wariantów algorytmu ewolucyjnego – SMS-EMOA, NSGA-II oraz dwóch wariantów pośrednich (SS-NSGA-II, GEN-SMS-EMOA)¹ – wraz z dedykowaną reprezentacją rozwiązania, operatorami genetycznymi i strategiami inicjalizacji populacji dostosowanymi do rozważanego problemu,
- weryfikację, czy hybrydowa strategia inicjalizacji populacji daje istotną przewagę nad inicjalizacją czysto losową,
- przeprowadzenie planu czynnikowego 2×2 rozkładającego porównywane algorytmy na dwa niezależne czynniki projektowe: architekturę (steady-state vs. generacyjna) oraz kryterium selekcji (wkład do hypervolume vs. crowding distance),
- niezależne automatyczne strojenie hiperparametrów każdego z porównywanych wariantów, tak aby porównanie nie było obciążone nierównym dostrojeniem,
- ablację operatorów mutacji na najlepszym dostrojonym wariantcie, wskazującą, które operatory są niezbędne dla jakości rozwiązań.

Zakres pracy. Praca obejmuje implementację i eksperymentalne porównanie czterech wariantów algorytmu ewolucyjnego na ośmiu instancjach testowych o zróżnicowanej charakterystyce. Rozważane zadanie dotyczy planowania tras zespołu UAV na poziomie kolejności odwiedzania regionów obszaru poszukiwań oraz przypisania regionów do poszczególnych dronów. Zakłada się, że obszar został wcześniej podzielony na regiony – etap ten realizowany jest przez odrębne narzędzia. Praca nie obejmuje sterowania lotem UAV na poziomie trajektorii ciągłej ani aspektów sprzętowych dronów.

Struktura pracy. Praca składa się z pięciu rozdziałów. Rozdział 1 przedstawia problem optymalizacji tras UAV w misjach SAR, jego klasyfikację oraz przegląd literatury dotyczącej ewolucyjnych algorytmów wielokryterialnych. Rozdział 2 opisuje szczegółowo implementację czterech porównywanych wariantów algorytmu w planie czynnikowym 2×2 , wraz z reprezentacją rozwiązania, funkcjami celu i operatorami genetycznymi. Rozdział 3 przedstawia metodologię badań w kolejności: instancje testowe, miarę jakości (hypervolume), narzędzia statystyczne oraz plan czterech eksperymentów. Rozdział 4 prezentuje i omawia wyniki przeprowadzonych eksperymentów. Rozdział 5 zawiera podsumowanie z syntezą wniosków, omówieniem ograniczeń pracy oraz wskazaniem kierunków dalszych badań.

¹ W dalszej części pracy nazwy SMS-EMOA, NSGA-II, SS-NSGA-II i GEN-SMS-EMOA oznaczają warianty z ujednoliconą, losową selekcją rodziców; różnice względem wersji kanonicznych omówiono w Sekcjach 2.6 i 5.4.

Rozdział 1

Problem i przegląd literatury

W niniejszym rozdziale przedstawiono kontekst badawczy pracy. Punktem wyjścia jest opis misji SAR i roli UAV jako narzędzia poszukiwawczego, po czym sformułowano problem optymalizacji tras UAV i sklasyfikowano go w hierarchii problemów kombinatorycznych. Następnie wprowadzono niezbędne pojęcia z zakresu optymalizacji wielokryterialnej i ewolucyjnych algorytmów wielokryterialnych. Rozdział kończy przegląd powiązanych prac i pozycjonowanie wkładu niniejszej pracy.

1.1. Misje poszukiwawczo-ratownicze i rola UAV

Misje poszukiwawczo-ratownicze (SAR) stanowią kluczowy element odpowiedzi na sytuacje kryzysowe takie jak trzęsienia ziemi, powodzie, lawiny śnieżne czy katastrofy budowlane. Ich głównym celem jest jak najszybsze zlokalizowanie osób poszkodowanych, ocena ich stanu oraz dostarczenie pomocy. Skuteczność operacji ratunkowych jest silnie zależna od czasu: pierwsze minuty i godziny po wystąpieniu zdarzenia mają znaczenie krytyczne dla szans przeżycia ofiar.

Tradycyjne metody prowadzenia misji SAR oparte na zespołach ratowniczych poruszających się pieszo lub pojazdami napotykają istotne ograniczenia w scenariuszach katastrofalnych. Drogi i mosty mogą być zniszczone w wyniku powodzi czy trzęsienia ziemi, teren bywa niedostępny dla pojazdów lądowych (np. w terenie górskim lub zalanym), a liczba ratowników w stosunku do rozmiaru obszaru poszukiwań jest niewystarczająca dla zapewnienia szybkiego przeszukania. Bezzałogowe statki powietrzne (UAV) stanowią rozwiązanie wielu spośród tych problemów [21].

Zastosowania UAV w SAR. UAV wyposażone w kamery wizyjne i termalne wykrywają poszkodowanych na podstawie obrazu lub sygnatur ciepła, a przy tym są tanie i mogą być wdrażane szybko, bez ryzyka dla pilota [21].

Misje wielodronowe. Pojedynczy UAV ma ograniczony zasięg, czas lotu i powierzchnię, jaką może przeszukać w ograniczonym czasie operacji. Z tego powodu współczesne podejścia do operacji SAR coraz częściej zakładają wykorzystanie zespołu wielu dronów działających równocześnie [21]. Wykorzystanie zespołu k UAV pozwala na zrównoleglenie pracy, co w idealnym przypadku skraca całkowity czas misji nawet k -krotnie, oraz na pokrycie większych obszarów. Zysk ten nie jest jednak automatyczny: ponieważ czas misji wyznacza najwolniejszy dron, nierównomierny podział obszaru między drony może zniwelować korzyść płynącą z samego zwiększenia ich liczby.

1.2. Pełne pokrycie i planowanie ścieżki UAV

Problem planowania ścieżek UAV w misjach SAR należy do klasy problemów planowania pełnego pokrycia (*Coverage Path Planning*, CPP). W ogólnym sformułowaniu CPP polega na wyznaczeniu trajektorii dla jednego lub wielu agentów (np. robotów lub dronów), która zapewnia odwiedzenie każdego punktu określonego obszaru przynajmniej raz, przy jednoczesnej optymalizacji wybranego kryterium jakości, np. czasu lub długości trasy [14, 4].

Dekompozycja obszaru. W praktyce bezpośrednie rozwiązywanie CPP dla obszarów o złożonej geometrii jest trudne obliczeniowo, dlatego standardowym podejściem jest dwuetapowa dekompozycja problemu [14, 4]. W pierwszym etapie obszar dzielony jest na mniejsze, zwykle wypukłe regiony (prostokątne lub trójkątne), dla których istnieją sprawdzone metody generowania tras pokrycia. W drugim etapie problem sprowadza się do wyznaczenia kolejności odwiedzania regionów przez każdego agenta oraz przypisania regionów do agentów – co stanowi problem optymalizacji kombinatorycznej, trudny obliczeniowo, choć o skończonej i dobrze zdefiniowanej przestrzeni rozwiązań. Niniejsza praca koncentruje się na tym drugim etapie, zakładając że dekompozycja obszaru jest już wyznaczona.

Specyfika SAR. Klasyczne sformułowanie CPP zakłada równoważność wszystkich fragmentów obszaru – celem jest jedynie odwiedzenie każdego punktu. W kontekście SAR założenie to nie jest jednak adekwatne: różne regiony obszaru katastrofy mogą charakteryzować się różną oczekiwaną liczbą ofiar (*Expected Number of Victims*, ENV) – wielkością zaczerpniętą z modelu, na którym opiera się niniejsza praca [27, 28] – wynikającą z gęstości zaludnienia, lokalizacji budynków mieszkalnych czy struktury terenu. Co więcej, kolejność odwiedzania regionów ma bezpośredni wpływ na skuteczność misji: odwiedzenie regionu o wysokiej populacji pod koniec misji oznacza, że osoby tam się znajdujące będą oczekiwać pomocy najdłużej. Z tego powodu w problemie SAR niezbędne jest uwzględnienie priorytetów regionów – regiony o wyższej oczekiwanej liczbie ofiar powinny być odwiedzane wcześniej w ramach misji. Prowadzi to do zasygnalizowanego już we Wstępie konfliktu między czasem misji a dynamiką odkrywania ofiar; wstępną charakterystykę obu kryteriów przedstawia Sekcja 1.3, a ich formalną definicję – Rozdział 2.

1.3. Sformułowanie problemu

Problem badany w niniejszej pracy stanowi rozszerzenie klasycznego CPP o elementy charakterystyczne dla misji SAR. Dane wejściowe problemu obejmują:

- obszar katastrofy podzielony na n regionów,
- dla każdego regionu – oczekiwaną liczbę ofiar oraz czas potrzebny na jego pełne przeszukanie,
- macierz czasów przelotu między każdą parą regionów oraz wektor czasów dolotu z bazy startowej do każdego regionu,

— liczbę dronów k uczestniczących w misji.

Celem optymalizacji jest wyznaczenie zestawu k tras – po jednej dla każdego drona – spełniających następujące warunki: każdy region zostanie odwiedzony przez dokładnie jednego drona (brak redundancji i zapewnienie kompletności pokrycia), a uzyskany zestaw tras stanowi dobry kompromis względem dwóch kryteriów, obu podlegających minimalizacji:

- **czas zakończenia misji** (f_1) – całkowity czas, po którym wszystkie drony zakończą swoje trasy,
- **współczynnik nieodkrytych ofiar** (f_2) – uśredniony w czasie udział ofiar jeszcze nieodnalezionych; im wcześniej ofiary są odnajdywane, tym wartość f_2 niższa.

Oba kryteria wyrażono tak, by podlegały minimalizacji: zamiast maksymalizować liczbę i szybkość odnalezień, minimalizujemy ich dopełnienie – udział ofiar nieodnalezionych. Ujednolicenie kierunku obu kryteriów jest wygodne przy późniejszej ocenie jakości wskaźnikiem hypervolume (Rozdział 3). Formalne definicje obu funkcji celu oraz reprezentacji rozwiązania przedstawiono w Rozdziale 2.

Specyfika rozważanego wariantu problemu. Sformułowanie powyżej jest zgodne z modelem zaproponowanym przez Trojanowskiego i in. [27, 28], ze szczególnym uwzględnieniem podziału obszaru na regiony o różnych priorytetach. Instancje testowe wykorzystane w niniejszej pracy reprezentują obszary katastrof o zróżnicowanej charakterystyce.

1.4. Klasyfikacja problemu

Po dekompozycji obszaru na regiony i sprowadzeniu zadania do wyznaczenia kolejności ich odwiedzania, rozważany problem można scharakteryzować jako wariant klasycznego problemu marszrutyzacji pojazdów (*Vehicle Routing Problem*, VRP) [19]. W klasycznym VRP zespół pojazdów dostarcza towary do zbioru klientów rozproszonych geograficznie, minimalizując całkowitą długość tras. W rozważanym wariantcie pojazdami są drony, klientami – regiony obszaru katastrofy, a zamiast dostarczania towarów wykonywana jest operacja przeszukiwania regionu.

Rozważany problem wyróżnia się względem klasycznego VRP dwiema cechami związanymi z kryterium optymalizacji: czas zakończenia misji najwolniejszego drona, tzw. *makespan* [24], zamiast łącznej długości tras, oraz drugie kryterium optymalizacji – współczynnik nieodkrytych ofiar.

Złożoność obliczeniowa. Klasyczny VRP należy do klasy problemów NP-trudnych [19]. W konsekwencji rozwiązanie dokładne problemu jest niewykonalne obliczeniowo dla instancji o praktycznych rozmiarach – w rozważanej pracy instancje testowe zawierają od 259 do 599 regionów, co daje astronomicznie dużą przestrzeń rozwiązań rzędu $n!$; pełny wzór, uwzględniający dodatkowo podział regionów między drony, podano w Sekcji 2.1. Z tego powodu praktyczne rozwiązanie problemu wymaga zastosowania metod

heurystycznych lub metaheurystycznych [10], które rezygnują z gwarancji optymalności w zamian za znacznie krótszy czas działania.

Algorytmy ewolucyjne stanowią jedną z najszerzej stosowanych klas metaheurystyk dla problemów typu VRP i ich wariantów [10], szczególnie w kontekście problemów wielokryterialnych, gdzie populacyjny charakter algorytmów ewolucyjnych pozwala naturalnie wyznaczać cały zbiór rozwiązań kompromisowych zamiast pojedynczego rozwiązania optymalnego [11].

1.5. Optymalizacja wielokryterialna

W problemach jednokryterialnych pojęcie rozwiązania optymalnego jest jednoznaczne – istnieje zwykle jedno rozwiązanie minimalizujące bądź maksymalizujące wartość funkcji celu. W problemach wielokryterialnych sytuacja jest bardziej złożona, ponieważ poszczególne kryteria mogą być ze sobą sprzeczne – poprawa jednego oznacza pogorszenie innych. W rozważanym problemie SAR sprzeczność występuje między dwoma minimalizowanymi kryteriami – czasem misji (f_1) a współczynnikiem nieodkrytych ofiar (f_2): premiowanie wczesnego odwiedzania regionów o wysokiej populacji może wymagać dłuższych przelotów, wydłużających misję.

Dominacja Pareto. Klasyczne pojęcie porównywania rozwiązań w optymalizacji wielokryterialnej oparte jest na *relacji dominacji Pareto* [11]. Rozwiązanie **a** dominuje rozwiązanie **b** (zapis: $\mathbf{a} \prec \mathbf{b}$) wtedy i tylko wtedy, gdy **a** nie jest gorsze od **b** pod względem żadnego kryterium i jest ściśle lepsze pod względem co najmniej jednego. Rozwiązanie nazywamy *niezdominowanym*, jeżeli nie istnieje żadne inne rozwiązanie, które by je dominowało.

Front Pareto. Zbiór wszystkich rozwiązań niezdominowanych w przestrzeni rozwiązań dopuszczalnych nazywamy *frontem Pareto*. Front Pareto reprezentuje wszystkie możliwe optymalne kompromisy między kryteriami – żadne rozwiązanie spoza frontu nie może być uznane za optymalne (zawsze istnieje rozwiązanie należące do frontu które jest co najmniej tak dobre we wszystkich kryteriach). Z praktycznego punktu widzenia front Pareto dostarcza zbioru alternatywnych rozwiązań, spośród których można wybrać to najlepiej odpowiadające priorytetom konkretnej misji.

Aproksymacja frontu Pareto. W praktyce wyznaczenie dokładnego frontu Pareto dla problemów NP-trudnych jest niewykonalne obliczeniowo. Stosowane metody dążą zatem do wyznaczenia *aproksymacji frontu Pareto* – zbioru rozwiązań niezdominowanych możliwie bliskich prawdziwemu frontowi i równomiernie rozmieszczonych wzdłuż niego. Dobra aproksymacja powinna spełniać dwa kryteria jakości [7, 11]: *zbieżność* (rozwiązania powinny leżeć jak najbliżej prawdziwego frontu Pareto) oraz *różnorodność* (rozwiązania powinny pokrywać cały zakres frontu, a nie skupiać się w jednym obszarze).

Wskaźniki jakości aproksymacji. Do oceny jakości aproksymacji frontu Pareto stosuje się różne wskaźniki ilościowe. Najczęściej używanym wskaźnikiem jest *hypervolume*

(*HV*) [32, 33] – miara objętości obszaru przestrzeni celów dominowanego przez zbiór rozwiązań i ograniczonego ustalonym z góry punktem referencyjnym. Wskaźnik ten jest ściśle zgodny z dominacją Pareto: jeżeli zbiór A dominuje zbiór B , to $HV(A) > HV(B)$ [13], co czyni go atrakcyjnym wyborem do porównań między różnymi aproksymacjami; własność ta wyróżnia *HV* spośród innych znanych wskaźników jakości [33] (Sekcja 3.2). W niniejszej pracy jako podstawową miarę jakości stosujemy *HV*, ze względu na jego szeroką akceptację w społeczności badawczej oraz zgodność z dominacją Pareto. Szczegółowy opis *HV* wraz z definicją formalną przedstawiono w Rozdziale 3 (Sekcja 3.2).

1.6. Ewolucyjne algorytmy wielokryterialne

Algorytmy ewolucyjne (*Evolutionary Algorithms*, EA) stanowią klasę metod metaheurystycznych inspirowanych biologicznymi procesami ewolucji. W ich centrum znajduje się *populacja* rozwiązań kandydujących, która iteracyjnie ewoluuje pod wpływem trzech operatorów inspirowanych genetyką: *selekcji* – premiującej rozwiązania lepsze przy wyborze rodziców i osobników pozostających w populacji, *krzyżowania* – łączącego fragmenty dwóch rozwiązań rodzicielskich w nowe rozwiązanie, oraz *mutacji* – wprowadzającej drobne losowe zmiany w pojedynczym rozwiązaniu. Populacyjna natura algorytmów ewolucyjnych sprawia, że są one szczególnie dopasowane do problemów wielokryterialnych: populacja jest w naturalny sposób w stanie reprezentować wiele różnych kompromisów jednocześnie, dążąc do aproksymacji całego frontu Pareto [10, 11].

Architektury algorytmów ewolucyjnych. W literaturze algorytmów ewolucyjnych wyróżnia się dwie podstawowe architektury różniące się sposobem aktualizacji populacji (istnieją również warianty pośrednie, np. $(\mu + \lambda)$ dla $1 < \lambda < \mu$):

- **Architektura generacyjna elitarna** $(\mu + \mu)$ – w każdym pokoleniu generowana jest pełna populacja μ potomków, a następnie nowa populacja wybierana jest z populacji łączonej 2μ (rodzice + potomkowie) metodą selekcji środowiskowej (w wariancie nieelitarnym potomkowie zastępowaliby rodziców w całości, bez łączenia; w niniejszej pracy stosowany jest wyłącznie wariant elitarny, zgodny z NSGA-II). Pojedyncza generacja w tym wariancie wymaga μ ewaluacji funkcji celu, a aktualizacja populacji następuje jednorazowo po zakończeniu generowania wszystkich potomków;
- **Architektura steady-state** $(\mu + 1)$ – w każdej iteracji generowany jest dokładnie jeden potomek, a populacja jest aktualizowana przez bezpośrednie zastąpienie jednego z osobników nowym (zwykle według wybranego kryterium gorszego dopasowania). Pojedyncza iteracja wymaga jednej ewaluacji funkcji celu, a populacja jest w ciągłej aktualizacji.

Architektura generacyjna jest historycznie wcześniejsza [10] i pozostaje dominująca wśród współczesnych algorytmów wielokryterialnych (*Multi-Objective Evolutionary Algorithm*, MOEA) [9], natomiast architektura steady-state oferuje szybszą propagację dobrych genów w populacji: nowo wygenerowany dobry osobnik może zostać wybrany jako rodzic już przy tworzeniu kolejnego potomka, podczas gdy w architekturze generacyjnej

staje się on dostępny jako rodzic dopiero w następnym pokoleniu, po wygenerowaniu wszystkich μ potomków. Z drugiej strony, generacyjna ewolucja zapewnia większą stabilność populacji w ramach pokolenia, co może sprzyjać eksploracji przestrzeni rozwiązań. Empiryczne porównania obu architektur w kontekście MOEA pokazują, że wybór architektury może istotnie wpływać na jakość wyników, przy czym korzystniejsza architektura zależy od problemu: np. Durillo i in. [9] stwierdzili przewagę wariantów steady-state na ciągłych problemach testowych (ZDT, DTLZ, WFG). Niniejsza praca dostarcza obserwacji dla problemu kombinatorycznego optymalizacji tras UAV; oba warianty porównano systematycznie w planie czynnikowym (Sekcja 3.4.2), a szczegółową dyskusję w kontekście literatury – w tym pełne zestawienie z wynikami Durillo i in. – przedstawiono w Sekcji 4.5.

Krótką historią algorytmów MOEA. Rozwój ewolucyjnych algorytmów wielokryterialnych (przegląd: Emmerich i Deutz [11]) przebiegał od pierwszych metod z lat osiemdziesiątych (VEGA Schaffera, z niezależnymi podpopulacjami dla każdego kryterium), przez algorytmy oparte na dominacji Pareto z lat dziewięćdziesiątych (MOGA, NSGA, NPGA), po klasyczne dziś algorytmy drugiej generacji z lat 2001–2002: NSGA-II [7] i SPEA2 [31], łączące wydajne sortowanie niezdominowane z mechanizmami zachowania różnorodności. SMS-EMOA [2] (2007) oparł selekcję środowiskową na bezpośredniej maksymalizacji hypervolume. Z nowszych metod na uwagę zasługują MOEA/D [30], rozkładający problem wielokryterialny na zbiór podproblemów skalarnych optymalizowanych równocześnie, oraz NSGA-III [6], rozszerzający NSGA-II na problemy o wielu kryteriach (cztery i więcej) za pomocą predefiniowanych punktów referencyjnych.

NSGA-II. *Non-dominated Sorting Genetic Algorithm II* [7] jest jednym z najbardziej rozpowszechnionych algorytmów ewolucyjnych wielokryterialnych [11]. NSGA-II wykorzystuje architekturę generacyjną i oparty jest na dwóch kluczowych mechanizmach. Pierwszym jest *szybkie sortowanie niezdominowane* (*fast non-dominated sort*) dzielące populację na fronty o rosnącej dominacji: front pierwszy zawiera rozwiązania niezdominowane w populacji, drugi front rozwiązania niezdominowane po usunięciu pierwszego frontu, i tak dalej. Drugim mechanizmem jest *crowding distance* – miara zatłoczenia używana do utrzymania różnorodności rozwiązań w ramach jednego frontu. Crowding distance osobnika obliczana jest jako suma znormalizowanych odległości między jego sąsiadami (w przestrzeni celów) wzdłuż każdego z kryteriów; osobnicy w zatłoczonych obszarach frontu mają niską wartość crowding distance, podczas gdy osobnicy w rzadko reprezentowanych obszarach otrzymują wartość wysoką. NSGA-II osiąga dobre wyniki dla szerokiego spektrum problemów przy stosunkowo małej złożoności obliczeniowej ($O(M\mu^2)$ na pokolenie, gdzie M to liczba kryteriów, a μ – rozmiar populacji), co czyni go standardowym punktem odniesienia w ocenie nowych algorytmów wielokryterialnych [11].

SPEA2. *Strength Pareto Evolutionary Algorithm 2* [31], zaproponowany przez Zitzlera, Laumannsa i Thielego, wyróżnia się utrzymywaniem zewnętrznego archiwum osobników niezdominowanych, które uczestniczy również w selekcji. W niniejszej pracy przywoływany jest przede wszystkim dlatego, że to jego zastosowano we wcześniejszej pracy nad wielokryterialnym wariantem rozważanego problemu [28], którą praca rozwija. Badane tu wa-

rianty także gromadzą rozwiązania w zewnętrznym archiwum, lecz pełni ono u nich rolę wyłącznie wynikową – w odróżnieniu od SPEA2 nie bierze udziału w selekcji (Rozdział 2).

SMS-EMOA. *S-Metric Selection Evolutionary Multi-Objective Algorithm* [2], opracowany przez Beume, Naujoks i Emmericha, stanowi nowsze podejście do MOEA wykorzystujące wskaźnik *hypervolume* bezpośrednio w mechanizmie selekcji. SMS-EMOA jest algorytmem steady-state ($\mu + 1$), w którym w każdej iteracji osobnik z najgorszego frontu o najmniejszym wkładzie do *hypervolume* jest usuwany z populacji. Wkład do *hypervolume* danego osobnika definiowany jest jako różnica między *HV* całego frontu a *HV* frontu po usunięciu tego osobnika – intuicyjnie jest to powierzchnia przestrzeni celów dominowana wyłącznie przez tego osobnika. Osobnik o najmniejszym wkładzie jest najmniej cennym członkiem frontu, ponieważ jego usunięcie najmniej zmniejsza całkowite *HV*.

Wykorzystanie *hypervolume* – wskaźnika ściśle zgodnego z dominacją Pareto – czyni SMS-EMOA teoretycznie atrakcyjnym wyborem, szczególnie dla problemów dwukryterialnych, gdzie obliczenie wkładów do *hypervolume* sprowadza się do obliczenia różnic między prostokątami przestrzeni celów i jest tanie obliczeniowo. Dla problemów o większej liczbie kryteriów koszt obliczenia *hypervolume* znacząco rośnie [2], przez co stosowanie SMS-EMOA przy wielu kryteriach staje się kosztowne obliczeniowo.

Dwa czynniki różnicujące oba algorytmy. Choć NSGA-II i SMS-EMOA opierają się na wspólnym mechanizmie sortowania niezdominowanego, różnią się architekturą (generacyjna w NSGA-II vs. steady-state w SMS-EMOA) oraz kryterium selekcji środowiskowej (crowding distance vs. wkład do *hypervolume*) – to właśnie te dwa czynniki bada niniejsza praca. Algorytmy te różnią się ponadto sposobem selekcji rodziców (kanoniczny NSGA-II stosuje turniej binarny); aby porównanie dotyczyło wyłącznie dwóch badanych czynników, selekcję tę ujednolicono we wszystkich wariantach (Rozdział 2). Porównanie SMS-EMOA i NSGA-II oraz rozłożenie dzielącej je różnicy na te dwa czynniki projektowe stanowi przedmiot niniejszej pracy; służy temu plan czynnikowy 2×2 , którego cztery warianty wprowadzono w Rozdziale 2, a metodologię – w Sekcji 3.4.2.

1.7. Powiązane prace i pozycjonowanie

Problematyka optymalizacji tras UAV w kontekście SAR jest przedmiotem rosnącego zainteresowania badawczego, co odzwierciedlają liczne prace przeglądowe [21, 4, 14]. Niniejsza praca bezpośrednio rozwija serię badań nad tym zagadnieniem prowadzonych przez Trojanowskiego, Mikitiuka, Grzeszczaka oraz Guinanda.

Trojanowski i in. 2023. W pracy Trojanowski i in. [27] zaproponowano model problemu CPP dla misji SAR uwzględniający podział obszaru na regiony o różnych priorytetach (proporcjonalnych do oczekiwanej liczby ofiar) oraz wprowadzono funkcję oceny opartą na liniowym modelu odkrywania ofiar w czasie skanowania regionu. Do rozwiązania problemu zastosowano *Local Search* z problemowo-specyficznymi operatorami perturbacji działającymi wewnątrz pojedynczej trasy oraz między trasami (w niniejszej pracy określanymi

jako *intra-route* i *inter-route*). Praca ta sformułowała problem w wersji jednokryterialnej, łącząc czas misji i dynamikę odkrywania ofiar w pojedynczą funkcję skalaryzacji.

Trojanowski i in. 2024. W kolejnej pracy Trojanowski i in. [28] zaproponowano wielokryterialne sformułowanie problemu, w którym czas misji i dynamika odkrywania ofiar traktowane są jako dwa niezależne kryteria optymalizacji. Do rozwiązania problemu zastosowano algorytm SPEA2 wzbogacony o problemowo-specyficzne operatory perturbacji. W części eksperymentalnej porównano SPEA2 z rozwiązaniami losowymi oraz z wynikami Local Search z pracy 2023, uzyskując poprawę długości rozwiązań do około 20%.

Pozycjonowanie niniejszej pracy. Niniejsza praca wpisuje się w nurt badań zapoczątkowany przez prace Trojanowskiego i in. [27, 28], rozwijając go w następujących kierunkach:

1. Porównanie czterech wielokryterialnych algorytmów ewolucyjnych na rozważanym problemie: SMS-EMOA i NSGA-II wraz z dwoma wariantami pośrednimi. SMS-EMOA stanowi punkt wyjścia niniejszej pracy w miejsce stosowanego we wcześniejszych badaniach SPEA2, ze względu na wykorzystanie hypervolume bezpośrednio w mechanizmie selekcji.
2. Przeprowadzenie systematycznego planu czynnikowego 2×2 , rozdzielającego architekturę (steady-state vs. generacyjna) od kryterium selekcji (wkład do hypervolume vs. crowding distance) – dwóch czynników, którymi SMS-EMOA i NSGA-II różnią się jednocześnie (Seksja 1.6).
3. Niezależne automatyczne strojenie hiperparametrów narzędziem irace [20] dla każdego z czterech wariantów – w celu zbadania wpływu strojenia na jakość poszczególnych wariantów oraz sprawdzenia, czy przewaga architektury utrzymuje się, gdy każdy wariant działa w swojej najlepszej konfiguracji (a nie wynika z arbitralnie przyjętych parametrów domyślnych).
4. Ablację operatorów mutacji na najlepszym dostrojonym wariantcie, wskazującą, które operatory są niezbędne dla jakości rozwiązań, a które można pominąć – analizę komplementarną wobec automatycznego strojenia.

Wyniki planu czynnikowego (Rozdział 4) wskazują na istotny wpływ wyboru między architekturą steady-state a generacyjną na jakość rozwiązań – czynnika, którego znaczenie wskazywali również Durillo i in. [9], choć dla problemów ciągłych zaobserwowali oni przewagę przeciwnego wariantu architektury. Niniejsza praca dostarcza konkretnej obserwacji dla problemu kombinatorycznego optymalizacji tras UAV w misjach SAR, gdzie literatura nie dawała dotychczas jednoznacznej odpowiedzi.

Rozdział 2

Algorytmy i implementacja

Rozdział przedstawia implementację czterech wariantów algorytmu ewolucyjnego tworzących plan czynnikowy 2×2 : dwóch znanych z literatury (SMS-EMOA, NSGA-II) oraz dwóch skonstruowanych na potrzeby tego planu (SS-NSGA-II, GEN-SMS-EMOA), dopełniających wszystkie kombinacje badanych czynników. Najpierw opisano elementy wspólne wszystkim wariantom (reprezentację rozwiązania, funkcje celu, strategię inicjalizacji, operator krzyżowania i operatory mutacji). Następnie wprowadzono plan czynnikowy 2×2 i opisano każdy z czterech wariantów – warianty pośrednie jako modyfikacje algorytmów kanonicznych różniące się jednym czynnikiem, co pozwala izolować wpływ poszczególnych komponentów. Rozdział zamykają szczegóły implementacyjne.

2.1. Reprezentacja rozwiązania

Rozwiązanie problemu optymalizacji tras UAV reprezentowane jest jako zestaw k sekwencji indeksów regionów, gdzie k oznacza liczbę dronów biorących udział w misji. Każdej sekwencji odpowiada trasa jednego drona – uporządkowana lista regionów, które ten dron odwiedzi w ramach misji. Formalnie, dla problemu o n regionach i k dronach rozwiązanie definiowane jest jako:

$$x = (\pi_1, \pi_2, \dots, \pi_k), \quad (2.1)$$

gdzie π_j jest sekwencją regionów przypisanych do j -tego drona, a każdy region należący do zbioru $\{0, 1, \dots, n-1\}$ pojawia się w dokładnie jednej z k sekwencji.

Przykład. Dla problemu z $n = 10$ regionów i $k = 3$ dronów rozwiązanie może wyglądać następująco:

$$x = ([5, 3, 1, 7], [2, 4, 8, 9], [0, 6]),$$

co oznacza, że pierwszy dron odwiedza kolejno regiony 5, 3, 1, 7, drugi dron regiony 2, 4, 8, 9, a trzeci regiony 0, 6. Schemat takiego podziału regionów na trasy dronów przedstawia Rysunek 2.1.

Pozycja bazy startowej. Baza, z której startują wszystkie drony, nie pojawia się jawnie w reprezentacji rozwiązania. Czas dolotu z bazy do pierwszego regionu trasy jest wyliczany na podstawie danych instancji testowej (macierze czasów dolotu) i uwzględniany w harmonogramie lotu. Czas powrotu z ostatniego regionu do bazy nie wchodzi natomiast do

funkcji celu f_1 : zgodnie z wzorem (2.3) czas zakończenia misji wyznacza moment zakończenia skanowania ostatniego regionu, ponieważ z perspektywy poszukiwawczo-ratowniczej istotny jest czas przeszukania całego obszaru, a nie czas powrotu dronów do bazy.

Ograniczenia rozwiązania. Aby rozwiązanie było poprawne, musi spełniać następujące warunki:

- zestaw zawiera dokładnie k sekwencji (po jednej na drona),
- każdy z n regionów pojawia się w dokładnie jednej sekwencji,
- suma długości wszystkich sekwencji wynosi n (kompletność pokrycia obszaru).

Sekwencja przypisana do drona może być pusta – oznacza to, że dany dron nie bierze aktywnego udziału w misji. Poprawność względem powyższych warunków sprawdzana jest po każdej operacji modyfikującej rozwiązanie (krzyżowanie, mutacja).

Charakterystyka przestrzeni rozwiązań. Wybrana reprezentacja należy do kategorii reprezentacji permutacyjnych z podziałem na grupy, stosowanej powszechnie w problemach typu VRP (*Vehicle Routing Problem*) z kryterium minimalizacji makespanu [26, 27]. Liczba możliwych rozwiązań dla problemu o n regionach i k dronach rośnie silniowo z n i jest znacznie większa niż liczba permutacji n -elementowych: każda permutacja indeksów regionów może zostać podzielona na k uporządkowanych podsekwencji (z dopuszczeniem pustych) na $\binom{n+k-1}{k-1}$ sposobów, co daje łącznie $n! \cdot \binom{n+k-1}{k-1}$ kombinacji. Dla instancji testowych rozważanych w pracy (Sekcja 3.1), gdzie n mieści się w zakresie 259–599 regionów, a $k = 3$, przestrzeń rozwiązań jest astronomicznie duża – już dla najmniejszej rozważanej instancji ($n = 259$) liczba możliwych rozwiązań wynosi w przybliżeniu 5×10^{518} . Ponieważ problem VRP należy do klasy NP-trudnych [19], dokładne rozwiązanie jest niewykonalne obliczeniowo dla rozmiarów rozważanych w pracy, co uzasadnia zastosowanie metod metaheurystycznych takich jak algorytmy ewolucyjne.

2.2. Funkcje celu

Optymalizacja w niniejszej pracy obejmuje dwa kryteria, jednocześnie minimalizowane: czas zakończenia misji (f_1) oraz współczynnik nieodkrytych ofiar (f_2), sprzeczne ze sobą w sposób opisany w Sekcji 1.5. Poniżej przedstawiono ich formalne definicje.

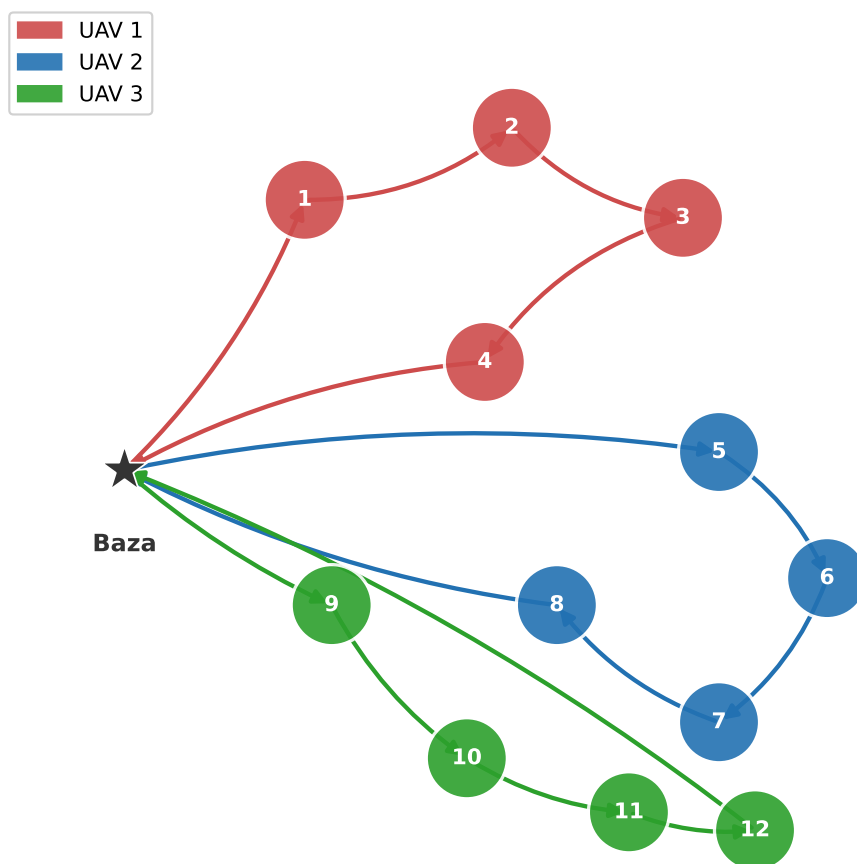
2.2.1. Harmonogram lotu

Obliczenie obu funkcji celu wymaga uprzedniego wyznaczenia *harmonogramu lotu* dla każdego drona. Harmonogram określa, w jakim momencie czasu dron dociera do kolejnych regionów i kiedy kończy ich skanowanie.

Dla trasy $\pi_j = (r_1, r_2, \dots, r_m)$ przypisanej do j -tego drona ($m = |\pi_j|$) niech τ_i oznacza czas przelotu do i -tego regionu trasy (z bazy dla $i = 1$, a z poprzedniego regionu dla $i > 1$), a σ_i – czas jego skanowania, zdefiniowany w instancji testowej:

$$\tau_i = \begin{cases} t_{\text{baza} \rightarrow r_1} & \text{dla } i = 1, \\ t_{r_{i-1} \rightarrow r_i} & \text{dla } i > 1, \end{cases} \quad \sigma_i = s_{r_i}.$$

Reprezentacja rozwiązania: 3 drony, 12 regionów
 Każda trasa startuje w bazie (powrót nie wlicza się do czasu misji)



Rysunek 2.1. Schemat reprezentacji rozwiązania jako zestawu tras UAV. Przykład dla problemu o $n = 12$ regionach i $k = 3$ dronach: każda trasa (czerwona, niebieska, zielona) startuje w bazie i odwiedza rozłączny podzbiór regionów. Powrót do bazy zaznaczono linią przerywaną, ponieważ nie jest on wliczany do czasu zakończenia misji (Sekcja 2.2.2). Regiony ponumerowano na rysunku od 1 do 12 dla czytelności; w formalnej reprezentacji (wzór (2.1)) indeksowane są od 0.

Harmonogram tworzony jest wówczas prostą rekurencją: czas przybycia do i -tego regionu to czas zakończenia skanowania poprzedniego regionu plus czas przelotu, a czas zakończenia jego skanowania to czas przybycia plus czas skanowania:

$$a_i = d_{i-1} + \tau_i, \quad d_i = a_i + \sigma_i, \quad i = 1, \dots, m \quad (d_0 := 0). \quad (2.2)$$

Całkowity czas zakończenia trasy j -tego drona oznaczamy $D_j := d_m$ (dla drona z pustą trasą przyjmuje się $D_j = 0$). Rozwijając rekurencję (2.2), D_j równa się po prostu sumie czasów przelotu i skanowania wzdłuż całej trasy, $D_j = \sum_{i=1}^m (\tau_i + \sigma_i)$ – zgodnie z intuicyjnym ujęciem czasu zakończenia trasy jako sumy czasu przelotów i czasu skanowania regionów, przyjętym również w pracy Trojanowskiego i in. [28].

2.2.2. Funkcja f_1 – czas zakończenia misji

Pierwsza funkcja celu definiowana jest jako moment, w którym ostatni z k dronów kończy skanowanie wszystkich przypisanych mu regionów:

$$f_1(x) = \max_{j \in \{1, \dots, k\}} D_j, \quad (2.3)$$

gdzie D_j (wzór (2.2)) oznacza czas zakończenia skanowania ostatniego regionu w trasie j -tego drona; dla drona z pustą trasą przyjmuje się $D_j = 0$, co nie wpływa na wynik maksimum, ponieważ taki dron nie wydłuża czasu misji.

Funkcja f_1 odpowiada *makespan* – klasycznej mierze czasu w problemach szeregowania równoległego [24]. Z perspektywy misji poszukiwawczo-ratowniczej f_1 wyraża czas, po którym obszar katastrofy zostaje w całości przeszukany. Minimalizacja f_1 premiuje równomierne obciążenie dronów – trasa najdłuższego drona wyznacza całkowity czas misji, więc skrócenie najdłuższej trasy wymaga przeniesienia części regionów do dronów mniej obciążonych.

2.2.3. Funkcja f_2 – współczynnik nieodkrytych ofiar

Druga funkcja celu mierzy, jak szybko ofiary są odkrywane podczas misji. Z perspektywy SAR znacznie korzystniej jest zlokalizować ofiary wcześniej, gdy istnieje większa szansa udzielenia im pomocy, niż późno – nawet jeśli całkowita liczba odkrytych ofiar jest identyczna.

Dynamikę tę ocenia się w oknie czasowym $[0, T_{\text{cut}}]$. W pracy Trojanowskiego i in. [28] oknem tym jest cała misja – ocena prowadzona jest do czasu jej zakończenia, czyli czasu najwolniejszego drona (w naszej notacji $T_{\text{cut}} = f_1$). Niniejsza praca skraca to okno do czasu zakończenia najszybszego drona:

$$T_{\text{cut}} = \min_{j \in \{1, \dots, k\}} D_j. \quad (2.4)$$

Powodem skrócenia jest to, że przy oknie sięgającym końca całej misji rozwiązania zdegenerowane – w których jeden dron ciągnie długi ogon złożony z nielicznych regionów, podczas gdy pozostałe drony odkryły już większość ofiar – otrzymują sztucznie zaniżoną (a więc korzystniejszą) wartość f_2 : przez cały ten długi ogon liczba odkrytych ofiar jest

już bliska całkowitej, więc okno oceny wypełnia się „za darmo”. Skrócenie okna do chwili, w której wszystkie drony jeszcze aktywnie pracują, usuwa tę premię. W minimum uwzględnia się przy tym wyłącznie trasy niepuste; w przypadku zdegenerowanym, gdy $T_{\text{cut}} \leq 0$ (brak niepuśczonego okna oceny), przyjmuje się $f_2 = 1$ – skoro w takim oknie nie sposób zaliczyć żadnego odkrycia, rozwiązanie otrzymuje wartość najgorszą.

Niech $V(t)$ oznacza skumulowaną liczbę ofiar odkrytych przez wszystkie drony do chwili t , a V_{total} całkowitą liczbę ofiar we wszystkich regionach. Misja idealna odkryłaby wszystkie ofiary natychmiast po rozpoczęciu, utrzymując $V(t) = V_{\text{total}}$ przez całe okno oceny; odkryte ofiary w czasie odpowiadają wówczas całe

$$\int_0^{T_{\text{cut}}} V_{\text{total}} dt = V_{\text{total}} \cdot T_{\text{cut}}. \quad (2.5)$$

Funkcję f_2 definiuje się jako znormalizowaną stratę względem tego ideału – czyli jaka część „ofiary w czasie” nie została odkryta w porównaniu do misji idealnej:

$$f_2(x) = 1 - \frac{\int_0^{T_{\text{cut}}} V(t) dt}{V_{\text{total}} \cdot T_{\text{cut}}}. \quad (2.6)$$

Wartość f_2 mieści się w zakresie $[0, 1]$, gdzie $f_2 = 0$ odpowiada idealnemu odkryciu wszystkich ofiar natychmiast po rozpoczęciu misji, a $f_2 = 1$ oznacza, że żadna ofiara nie została odkryta przed chwilą T_{cut} . Im niższa wartość f_2 , tym lepsza dynamika odkrywania ofiar.

Wkład pojedynczego regionu. Przyjęto, że ofiary regionu są odkrywane stopniowo podczas skanowania, a nie skokowo w momencie jego zakończenia: dla regionu skanowanego w przedziale $[a, d]$ i zawierającego V ofiar skumulowana liczba odkrytych ofiar rośnie liniowo od 0 w chwili a do V w chwili d , po czym pozostaje na poziomie V . Model liniowy przyjęto za Trojanowskim i in. [27] jako bardziej realistyczny od skokowego. Wkład takiego regionu do całki w liczniku wyrażenia (2.6) zależy więc od tego, jaka część jego okna skanowania mieści się przed T_{cut} :

$$w = \begin{cases} V \left(T_{\text{cut}} - \frac{a+d}{2} \right) & \text{skan zakończony } (d \leq T_{\text{cut}}), \\ \frac{V (T_{\text{cut}} - a)^2}{2(d-a)} & \text{skan trwa } (a < T_{\text{cut}} < d), \\ 0 & \text{skan nierozpoczęty } (a \geq T_{\text{cut}}). \end{cases} \quad (2.7)$$

Ograniczenie przyjętego okna. Skrócenie okna do czasu najszybszego drona ma też własną cenę: ocena pomija ofiary lokalizowane po jego zakończeniu, więc rozwiązania, w których jeden dron kończy bardzo wcześnie, mają krótsze okno oceny. Wybór między $T_{\text{cut}} = f_1$ a $T_{\text{cut}} = \min_j D_j$ jest zatem kompromisem między dwoma rodzajami zniekształceń. W pracy przyjęto definicję $T_{\text{cut}} = \min_j D_j$, uznając usunięcie premii dla rozwiązań z długim ogonem (opisane wyżej) za ważniejsze niż pominięcie ofiar odnajdywanych po zakończeniu najszybszego drona.

2.3. Inicjalizacja populacji

Jakość populacji początkowej istotnie wpływa na zbieżność algorytmu ewolucyjnego [10]; wpływ samej strategii inicjalizacji zbadano osobno w eksperymencie wstępnym (Sekcja 4.1). W niniejszej pracy zastosowano inicjalizację hybrydową, łączącą cztery strategie generowania osobników startowych o komplementarnych właściwościach (w nawiasach domyślny udział w populacji):

- **Najbliższego sąsiada** (20%) – każdy dron buduje trasę zachłannie: startuje od nieprzypisanego regionu o najkrótszym dolocie z bazy, a następnie dokłada kolejno region położony najbliżej ostatnio dodanego. Dla pierwszych $k - 1$ dronów długość trasy ograniczono do $\lfloor n/k \rfloor$ regionów (ostatni dron otrzymuje resztę), by jeden dron nie przejął większości regionów. Sprzyja niskiej wartości f_1 , nie uwzględnia liczby ofiar.
- **Klastrowa** (10%) – regiony sortowane po czasie dolotu z bazy i dzielone na k grup o zbliżonej liczności, przydzielanych dronom w losowej kolejności; wewnątrz grupy porządek wyznacza reguła najbliższego sąsiada. Dąży do geograficznego rozdzielania pól działania dronów.
- **Priorytetowa** (10%) – ukierunkowana na f_2 : regiony sortowane malejąco po oczekiwanej liczbie ofiar i przydzielane dronom cyklicznie, po jednym na drona, a wewnątrz każdej trasy również malejąco po liczbie ofiar, tak by dron zaczynał od regionu o najwyższym priorytecie. Promuje wczesne odkrywanie ofiar kosztem długości trasy.
- **Zrównoważona losowa** (60%) – regiony ustawiane w losowej kolejności i rozdzielane cyklicznie między drony (długości tras różnią się co najwyżej o jeden region), z dodatkowym losowym przetasowaniem wewnątrz trasy. Nie wykorzystuje wiedzy o problemie; jako komponent większościowy zapewnia różnorodność populacji dla operatorów krzyżowania i mutacji.

Przyjęte proporcje (większość – 60% – osobników losowych, mniejszość heurystycznych) mają łączyć różnorodność populacji startowej, kluczową dla operatorów genetycznych i zapewnianą przez komponent losowy, z obecnością dobrych rozwiązań wyjściowych dostarczanych przez strategie heurystyczne. Podział ten dobrano na podstawie tych przesłanek, a nie systematycznego strojenia; eksperyment wstępny (Sekcja 4.1) potwierdza jedynie przewagę inicjalizacji hybrydowej nad czysto losową, nie zaś optymalność samych proporcji (por. ograniczenia w Sekcji 5.4).

2.3.1. Perturbacja osobników heurystycznych

Trzy strategie heurystyczne (najbliższego sąsiada, klastrowa, priorytetowa) generują osobników w sposób deterministyczny lub częściowo deterministyczny, co przy generowaniu wielu osobników tego samego typu prowadziłoby do nadmierowej koncentracji populacji wokół wąskiego obszaru przestrzeni rozwiązań. Aby temu przeciwdziałać, każdy osobnik wygenerowany jedną z trzech strategii heurystycznych jest następnie poddawany losowej perturbacji w postaci dziesięciu wymian wewnątrz losowo wybranych tras. Liczbę tę dobrano wstępnie i nie była ona przedmiotem strojenia irace (por. ograniczenia w Sekcji 5.4). Po każdej perturbacji sprawdzana jest poprawność rozwiązania.

2.4. Operator krzyżowania

Do krzyżowania osobników zastosowano *Order Crossover* (OX) [5], operator zaprojektowany dla rozwiązań reprezentowanych jako permutacje. Klasyczny OX działa na pojedynczej permutacji, natomiast rozwiązanie w niniejszej pracy składa się z k sekwencji (po jednej na drona). Poniżej opisano najpierw klasyczny operator, a następnie jego adaptację do rozwiązań wielotrasowych (Sekcja 2.4.1).

Klasyczny OX dla dwóch permutacji P_1 i P_2 tej samej długości n działa w dwóch krokach. Najpierw kopiuje do potomka ciągły segment permutacji P_1 (wyznaczony dwoma losowymi punktami cięcia). Pozostałe pozycje wypełnia następnie elementami rodzica P_2 nieobecnymi w skopiowanym segmencie, w kolejności, w jakiej występują one w P_2 . Dzięki temu potomek dziedziczy segment w całości z P_1 , a dla reszty regionów zachowuje względną kolejność z P_2 – co czyni OX odpowiednim dla problemów, w których istotna jest kolejność odwiedzania węzłów.

Przykład (OX klasyczny). Dla rodziców $P_1 = [1, 2, 3, 4, 5, 6]$ oraz $P_2 = [4, 1, 6, 2, 5, 3]$ i segmentu skopiowanego z P_1 na pozycjach 3–4 ($[3, 4]$) potomek to $[1, 6, 3, 4, 2, 5]$: zachowuje segment $[3, 4]$ z P_1 , a pozostałe regiony $(1, 6, 2, 5)$ przyjmuje w kolejności ich występowania w P_2 .

2.4.1. Adaptacja OX dla wielu UAV

Aby zastosować operator OX do rozwiązań składających się z k ścieżek, wykorzystano reprezentację typu *giant tour* – pojedynczą permutację wszystkich regionów, bez separatorów oddzielających trasy poszczególnych dronów – stosowaną przez Prinsa [26] w algorytmach ewolucyjnych dla VRP. Procedura adaptacji obejmuje trzy kroki:

1. **Konkatenacja:** ścieżki każdego z rodziców łączone są w jedną sekwencję długości n , a pozycje granic między ścieżkami zapamiętywane są jako punkty podziału.
2. **OX:** operator Order Crossover stosowany jest do połączonych permutacji obu rodziców.
3. **Rekonstrukcja:** potomna permutacja długości n dzielona jest z powrotem na k ścieżek przy użyciu punktów podziału losowo wybranego rodzica (z równym prawdopodobieństwem).

Losowy wybór punktów podziału z jednego z rodziców sprawia, że potomek dziedziczy zarówno wzorzec sekwencji odwiedzania regionów (z OX), jak i wzorzec podziału regionów między drony (z jednego z rodziców). Rekonstrukcja różni się w tym punkcie od podejścia Prinsa [26], w którym permutację dzieli się na trasy za pomocą optymalnej procedury *Split* (programowanie dynamiczne): tutaj podział dziedziczony jest bezpośrednio po rodzicu, dzięki czemu staje się on – obok kolejności regionów – osobną cechą podlegającą krzyżowaniu, bez dodatkowego kosztu obliczeniowego. Po rekonstrukcji sprawdzana jest poprawność rozwiązania; jeśli została naruszona, zwracana jest kopia pierwszego rodzica.

Algorithm 1 Order Crossover dla wielu UAV**Require:** Rozwiązania rodzicielskie $x_1 = (\pi_1, \dots, \pi_k)$, $x_2 = (\pi'_1, \dots, \pi'_k)$ **Ensure:** Rozwiązanie potomne o

- 1: $F_1, S_1 \leftarrow$ konkatenacja tras x_1 ▷ permutacja + granice tras
- 2: $F_2, S_2 \leftarrow$ konkatenacja tras x_2
- 3: $F_o \leftarrow \text{OX}(F_1, F_2)$
- 4: $S \leftarrow$ losowo wybrane S_1 lub S_2 ▷ równe prawdopodobieństwo
- 5: $o \leftarrow$ podział F_o według granic S
- 6: **if** o nie jest poprawne **then**
- 7: **return** kopia x_1
- 8: **end if**
- 9: **return** o

2.5. Operatory mutacji

W algorytmie wykorzystano sześć operatorów mutacji, dzielących się na *intra-route* – modyfikujące kolejność regionów w ścieżce jednego drona – oraz *inter-route*, restrukturyzujące podział regionów między drony (a więc balans obciążenia). Przy tworzeniu każdego osobnika potomnego algorytm losowo wybiera jeden operator zgodnie z przyjętym rozkładem wag (Tabela 3.2 podaje wagi domyślne; Tabela 4.4 w Rozdziale 4 – dostrojone). Sześć operatorów zestawia Tabela 2.1.

Tabela 2.1. Operatory mutacji zastosowane w algorytmie

Operator	Typ	Działanie
<i>swap</i>	intra	zamienia miejscami dwa regiony w jednej ścieżce
<i>invert</i>	intra	odwraca kolejność losowego segmentu ścieżki (ruch 2-opt [10])
<i>or-opt</i>	intra/inter	przenosi spójny blok $\ell \in \{1, 2, 3\}$ regionów w inne miejsce tej samej lub innej ścieżki (forma 3-opt [23, 26])
<i>transfer</i>	inter	przenosi jeden region do innej ścieżki (balansuje obciążenie dronów)
<i>exchange</i>	inter	wymienia po jednym regionie między dwiema ścieżkami (1:1)
<i>cross-route</i>	inter	wymienia końcowe fragmenty dwóch ścieżek (ruch 2-opt* [25])

Przykłady działania operatorów. Poniżej zilustrowano działanie każdego operatora na małym przykładzie (regiony oznaczono liczbami; dla operatorów *inter* pokazano dwie ścieżki dronów rozdzielone symbolem $\parallel$):

- *swap*: $[1, 2, 3, 4] \rightarrow [1, 4, 3, 2]$ – zamiana regionów 2 i 4;
- *invert*: $[1, 2, 3, 4, 5] \rightarrow [1, 4, 3, 2, 5]$ – odwrócenie segmentu $[2, 3, 4]$;
- *or-opt*: $[1, 2, 3, 4, 5] \rightarrow [1, 4, 2, 3, 5]$ – przeniesienie bloku $[2, 3]$ za region 4;
- *transfer*: $[1, 2, 3] \parallel [4, 5] \rightarrow [1, 3] \parallel [4, 2, 5]$ – region 2 do drugiej ścieżki;
- *exchange*: $[1, 2, 3] \parallel [4, 5] \rightarrow [1, 5, 3] \parallel [4, 2]$ – wymiana regionów 2 i 5;
- *cross-route*: $[1, 2|3] \parallel [4|5, 6] \rightarrow [1, 2, 5, 6] \parallel [4, 3]$ – wymiana końcówek za kre-ską $|$.

2.5.1. Procedura mutacji

Każdy osobnik potomny poddawany jest mutacji z prawdopodobieństwem p_m . W razie mutacji losowany jest jeden z sześciu operatorów z Tabeli 2.1, z prawdopodobieństwem proporcjonalnym do jego wagi w_i (parametru konfiguracji algorytmu). Nadanie operatorowi wagi 0 wyłącza go z losowania, co pozwala badać mniejsze zestawy operatorów bez zmiany samej procedury. Po zastosowaniu operatora sprawdzana jest poprawność rozwiązania; jeśli operator ją naruszył, zwracany jest niezmienny osobnik wyjściowy.

Algorithm 2 Procedura mutacji

Require: Rozwiązanie x , prawdopodobieństwo mutacji p_m , operatory mutacji $\mathcal{O}_1, \dots, \mathcal{O}_6$ z wagami $w_1, \dots, w_6$

Ensure: Zmutowane rozwiązanie x'

- 1: **if** $\text{rand}() > p_m$ **then**
 - 2: **return** x ▷ brak mutacji
 - 3: **end if**
 - 4: Wylosuj operator $o \in \{\mathcal{O}_1, \dots, \mathcal{O}_6\}$ z pr. $\propto w_o$
 - 5: $x' \leftarrow o(x)$
 - 6: **if** x' nie jest poprawne **then**
 - 7: **return** x ▷ cofnij mutację
 - 8: **end if**
 - 9: **return** x'
-

2.6. Plan czynnikowy: cztery warianty algorytmu

W celu zbadania, które komponenty projektowe odpowiadają za jakość uzyskiwanych wyników, w pracy zaimplementowano cztery warianty algorytmu ewolucyjnego, różniące się dwoma czynnikami projektowymi: architekturą (steady-state vs. generacyjna) oraz kryterium selekcji środowiskowej (wkład do hypervolume vs. crowding distance). Cztery kombinacje dwóch czynników po dwóch poziomach tworzą plan czynnikowy 2×2 (Rozdział 3, Sekcja 3.4.2):

- **SMS-EMOA** – steady-state + wkład do HV (Sekcja 2.7),

- **NSGA-II** – generacyjny + crowding distance (Sekcja 2.8),
- **SS-NSGA-II** – steady-state + crowding distance (Sekcja 2.9),
- **GEN-SMS-EMOA** – generacyjny + wkład do HV (Sekcja 2.10).

Wszystkie cztery warianty współdzielą reprezentację rozwiązania (Sekcja 2.1), funkcje celu (Sekcja 2.2), strategię inicjalizacji populacji (Sekcja 2.3), operator krzyżowania (Sekcja 2.4) oraz zestaw operatorów mutacji (Sekcja 2.5).

Archiwum rozwiązań niezdominowanych. Równolegle z populacją roboczą P każdy z czterech wariantów utrzymuje archiwum E gromadzące rozwiązania niezdominowane napotkane w trakcie przebiegu (a nie tylko obecne w aktualnej populacji). Aktualizacja E polega na połączeniu go z frontem niezdominowanym bieżącej populacji, deduplikacji po wartościach (f_1, f_2) i ponownym sortowaniu niezdominowanym, tak że w E pozostają wyłącznie rozwiązania niezdominowane. W pseudokodach (Alg. 3–4) krok ten pokazano konceptualnie w pętli głównej; w implementacji archiwum odświeżane jest okresowo (co 10 generacji) oraz na zakończenie przebiegu. Częstotliwość tej aktualizacji nie wpływa istotnie na raportowane wartości HV : o wyniku decyduje front populacji końcowej, zawsze uwzględniany przy aktualizacji finalnej. Archiwum stanowi finalny wynik każdego wariantu.

Zewnętrzne archiwum rozwiązań niezdominowanych to technika znana z klasycznych algorytmów wielokryterialnych, m.in. z SPEA2 [31] (zastosowanego we wcześniejszej pracy nad tym problemem [28]). Zachodzi jednak istotna różnica: w SPEA2 archiwum uczestniczy w selekcji i przypisaniu przystosowania, natomiast w niniejszej pracy pełni rolę wyłącznie wynikową – gromadzi rozwiązania, z których liczone są raportowane wartości HV , i nie wpływa na przebieg samego algorytmu.

Aby porównanie izolowało wyłącznie dwa badane czynniki, selekcję rodziców ujednolicono we wszystkich czterech wariantach do wyboru losowego (dwa osobniki z populacji z rozkładem jednostajnym). Jest to celowa *zmienna kontrolowana*: skoro sposób selekcji rodziców jest identyczny w każdym wariancie, obserwowane różnice można przypisać architekturze i kryterium selekcji środowiskowej, a nie selekcji rodziców. Wybór losowy nie jest przy tym sprzeczny z pracą opisującą SMS-EMOA [2] (która nie narzuca mechanizmu selekcji rodziców), odchodzi natomiast od kanonicznego NSGA-II, stosującego selekcję turniejową. Wariant oznaczany w pracy jako „NSGA-II” odpowiada więc kanonicznemu NSGA-II architekturą i kryterium selekcji środowiskowej, lecz nie sposobem selekcji rodziców; wniosków dotyczących tego wariantu nie należy zatem wprost przenosić na jego wersję kanoniczną (por. Sekcja 5.4).

2.7. SMS-EMOA

Punktem wyjścia niniejszej pracy jest algorytm SMS-EMOA (*S-Metric Selection Evolutionary Multi-Objective Algorithm*) zaproponowany przez Beume, Naujoks i Emmericha [2] – wielokryterialny algorytm ewolucyjny o architekturze steady-state ($\mu+1$), wyróżniający się wykorzystaniem *wkładu do hypervolume* jako kryterium selekcji środowiskowej. W każdej iteracji generowany jest jeden potomek, po czym populacja $\mu+1$ dzielona jest na

fronty niezdominowane (*fast non-dominated sort* [7]) i z najgorszego frontu usuwany jest osobnik o najmniejszym wkładzie do hypervolume – powierzchni przestrzeni celów zdominowanej wyłącznie przez niego. Wartości funkcji celu normalizuje się przy tym względem zakresu całej populacji $\mu + 1$. Pełny algorytm przedstawiono w Pseudokodzie 3; korzysta on ze wspólnego archiwum rozwiązań niezdominowanych E (Sekcja 2.6).

Algorithm 3 SMS-EMOA (steady-state, selekcja wg wkładu do HV)

Require: Rozmiar populacji μ , budżet N_{eval} , p_c , p_m , wagi mutacji $w_1, \dots, w_6$

Ensure: Archiwum rozwiązań niezdominowanych E

```

1:  $P \leftarrow$  inicjalizacja hybrydowa ( $\mu$  osobników); oceń;  $n \leftarrow \mu$ 
2:  $E \leftarrow$  front niezdominowany z  $P$ 
3: while  $n < N_{\text{eval}}$  do
4:    $c \leftarrow$  wariacja( $P$ )  $\triangleright$  OX + mutacja; Alg. 1, 2
5:   oceń  $c$ ;  $n \leftarrow n + 1$ ;  $P \leftarrow P \cup \{c\}$   $\triangleright |P| = \mu + 1$ 
6:    $F_v \leftarrow$  najgorszy front populacji  $P$ 
7:    $r \leftarrow \arg \min_{s \in F_v} [HV(F_v) - HV(F_v \setminus \{s\})]$   $\triangleright$  min. wkład do  $HV$ 
8:    $P \leftarrow P \setminus \{r\}$ ; aktualizuj archiwum  $E$   $\triangleright |P| = \mu$ 
9: end while
10: return  $E$ 

```

Złożoność obliczeniowa. W każdej iteracji dominują dwie operacje: sortowanie niezdominowane, o złożoności $O(M\mu^2)$ (M – liczba kryteriów, μ – rozmiar populacji), oraz obliczenie wkładów do hypervolume. Dla problemu dwukryterialnego ($M = 2$) to drugie prowadzi się do posortowania ostatniego frontu i obliczenia pól prostokątów ($O(|L| \log |L|)$, $|L|$ – rozmiar frontu), więc łączny koszt iteracji wynosi $O(\mu^2)$. Dla większej liczby kryteriów koszt hypervolume znacząco rośnie, co czyni SMS-EMOA kosztownym obliczeniowo przy wielu kryteriach [2].

2.8. NSGA-II

NSGA-II (*Non-dominated Sorting Genetic Algorithm II*) zaproponowany przez Deba i in. [7] to klasyczny algorytm wielokryterialny o architekturze generacyjnej ($\mu + \mu$) i kryterium selekcji opartym na crowding distance. W każdej generacji tworzona jest cała pula μ potomków, po czym z populacji łączonej 2μ wybieranych jest μ osobników do następnego pokolenia: włączane są kolejne pełne fronty od najlepszego, a ostatni częściowy front docinany jest po malejącej crowding distance – mierze zatłoczenia otoczenia osobnika w przestrzeni celów, sprzyjającej zachowaniu rozwiązań w rzadziej pokrytych obszarach. Selekcję rodziców, jak w pozostałych wariantach, ujednolicono do wyboru losowego (por. Sekcja 2.6). Złożoność jednej generacji wynosi $O(M\mu^2)$, zdominowana – podobnie jak w SMS-EMOA – przez sortowanie niezdominowane [7]. Sortowanie to wykonuje się jednak raz na μ potomków, a nie po każdym potomku jak w architekturze steady-state, więc koszt przypadający na pojedynczą ewaluację funkcji celu jest w wariancie generacyjnym odpowiednio niższy.

Algorithm 4 NSGA-II (generacyjny, selekcja wg crowding distance)**Require:** Rozmiar populacji μ , budżet N_{eval} , p_c , p_m , wagi mutacji $w_1, \dots, w_6$ **Ensure:** Archiwum rozwiązań niezdominowanych E

```

1:  $P \leftarrow$  inicjalizacja hybrydowa ( $\mu$  osobników); oceń;  $n \leftarrow \mu$ 
2:  $E \leftarrow$  front niezdominowany z  $P$ 
3: while  $n < N_{\text{eval}}$  do
4:    $Q \leftarrow \mu$  potomków z  $P$  (wariacja); oceń;  $n \leftarrow n + \mu$  ▷ OX + mutacja
5:    $\{F_1, F_2, \dots\} \leftarrow$  fronty niezdominowane  $P \cup Q$  ▷  $|P \cup Q| = 2\mu$ 
6:    $P \leftarrow$  pełne fronty  $F_1, F_2, \dots$ , póki  $|P| + |F_i| \leq \mu$ 
7:   dopełnij  $P$  z  $F_i$  wg największej crowding distance
8:   aktualizuj archiwum  $E$ 
9: end while
10: return  $E$ 

```

2.9. SS-NSGA-II

SS-NSGA-II (*Steady-state NSGA-II*), zaproponowany przez Durillo i in. [9], łączy architekturę steady-state z kryterium selekcji crowding distance. W kontekście niniejszej pracy wariant ten pozwala izolować wpływ kryterium selekcji przy ustalonej architekturze steady-state.

Algorytm jest identyczny z SMS-EMOA (Algorytm 3) z jedną różnicą: w kroku selekcji środowiskowej zamiast osobnika o najmniejszym wkładzie do hypervolume usuwany jest osobnik o najmniejszej crowding distance spośród ostatniego frontu.

2.10. GEN-SMS-EMOA

GEN-SMS-EMOA (*Generational SMS-EMOA*) – wariant utworzony na potrzeby planu czynnikowego – łączy architekturę generacyjną NSGA-II z kryterium wkładu do hypervolume z SMS-EMOA. Wariant ten dopełnia plan czynnikowy 2×2 , umożliwiając izolację wpływu architektury przy ustalonym kryterium selekcji opartym na hypervolume.

Pętla algorytmu jest identyczna z NSGA-II (Algorytm 4). Różnica polega na docinaniu częściowego ostatniego frontu: w NSGA-II ostatni front sortowany jest jednorazowo po malejącej crowding distance, natomiast w GEN-SMS-EMOA stosowane jest *iteracyjne* docinanie po wkładzie do hypervolume. W każdej iteracji obliczany jest wkład HV dla wszystkich pozostałych kandydatów, usuwany jest osobnik o najmniejszym wkładzie, a następnie wkłady są przeliczane. Iteracyjny charakter procedury wynika z tego, że wkład danego osobnika do HV zależy od jego sąsiadów w przestrzeni celów – usunięcie jednego osobnika zmienia wkłady pozostałych. Crowding distance, jako miara lokalna, może być obliczona raz i zastosowana w jednym przebiegu.

2.11. Podsumowanie wariantów

Tabela 2.2 podsumowuje cztery warianty wraz z ich kluczowymi charakterystykami implementacyjnymi.

Tabela 2.2. Cztery warianty algorytmu ewolucyjnego z planu czynnikowego 2×2

Wariant	Architektura	Selekcja środowiskowa
SMS-EMOA	steady-state ($\mu + 1$)	wkład do hypervolume
NSGA-II	generacyjna ($\mu + \mu$)	crowding distance
SS-NSGA-II	steady-state ($\mu + 1$)	crowding distance
GEN-SMS-EMOA	generacyjna ($\mu + \mu$)	wkład do hypervolume

Wybór tych konkretnie czterech wariantów wynika z planu czynnikowego 2×2 opisanego w Rozdziale 3 (Sekcja 3.4.2). Pozwala on jednocześnie ocenić, który z dwóch czynników projektowych – architektura czy kryterium selekcji – ma większy wpływ na jakość uzyskiwanych rozwiązań.

2.12. Szczegóły implementacyjne

Wszystkie cztery warianty algorytmu zaimplementowano w języku Python 3.12. Kluczowe komponenty algorytmów – struktura osobnika, operatory genetyczne, strategie inicjalizacji oraz pętle ewolucyjne – zaimplementowano od podstaw. Z bibliotek zewnętrznych wykorzystano NumPy do operacji numerycznych, SciPy do testów statystycznych, Matplotlib do wizualizacji wyników oraz pymoo jako niezależny kalkulator wskaźnika hypervolume do weryfikacji poprawności własnych obliczeń. Strojenie hiperparametrów (Sekcja 3.4.3) przeprowadzono pakietem `irace` w języku R.

Instancje testowe oraz wyniki każdego uruchomienia (konfiguracja, historia przebiegu, archiwum rozwiązań niezdominowanych) zapisywane są w formacie JSON. Wszystkie eksperymenty przeprowadzono na komputerze osobistym z procesorem Intel Core i5-3570K (4 rdzenie, 3,40 GHz) oraz 32 GB pamięci RAM. Pełny kod źródłowy wraz ze szczegółowym opisem struktury repozytorium i sposobu uruchamiania eksperymentów przedstawiono w Dodatku A; kod dostępny jest publicznie w repozytorium GitHub¹.

¹ <https://github.com/adak101/sms-emoa-uav-path-planning>

Rozdział 3

Metodologia badań

Rozdział opisuje metodologię części eksperymentalnej. Najpierw przedstawiono aparat pomiarowo-analityczny: charakterystykę instancji testowych (Sekcja 3.1), miarę jakości zbiorów rozwiązań niezdominowanych (Sekcja 3.2) oraz narzędzia statystyczne służące ocenie istotności różnic (Sekcja 3.3). Następnie omówiono cztery eksperymenty obliczeniowe (Sekcja 3.4), z których każdy opisano łącznie z metodą analizy jego wyników.

3.1. Instancje testowe

Eksperymenty przeprowadzono na zbiorze ośmiu instancji testowych reprezentujących różnorodne scenariusze obszarów katastrof, zróżnicowanych pod względem rozmiaru i liczby ofiar. Każda instancja zdefiniowana jest przez plik JSON zawierający: liczbę regionów n , liczbę ofiar w każdym regionie, czasy skanowania regionów, macierz czasów przelotu między regionami oraz czasy dolotu z bazy do każdego regionu.

Instancje podzielono na dwie grupy reprezentujące odmienne charakterystyki obszarów przeszukiwania, różniące się przede wszystkim skalą populacji ofiar:

- **TC-PGI** – cztery instancje (tcB, tcT, tcV, tcW) o mniejszych populacjach ofiar, wahających się od ok. 60 tys. do 222 tys.;
- **TC-SP** – cztery instancje (tcGB, tcGL, tcLO, tcSW) o znacząco większych populacjach, w zakresie od ok. 844 tys. do 1,85 mln ofiar.

Tabela 3.1 zestawia charakterystyki wszystkich ośmiu instancji testowych.

W dalszej części pracy instancje określane są jako trudniejsze lub łatwiejsze w sensie czysto operacyjnym: trudniejsza jest ta, na której algorytmy osiągają niższe wartości HV przy wyższym odchyleniu standardowym wyników.

Liczba UAV we wszystkich eksperymentach jest stała i wynosi $k = 3$. Zróżnicowanie instancji pod względem rozmiaru (od 259 do 599 regionów) oraz liczby ofiar (od ok. 60 tys. do 1,85 mln) umożliwia ocenę zachowania algorytmów w różnych warunkach problemowych.

3.2. Hypervolume jako miara jakości

Do oceny jakości zbioru rozwiązań niezdominowanych zastosowano wskaźnik *hypervolume* (HV) [32, 13] – jeden z najczęściej stosowanych wskaźników jakości w optymalizacji wielokryterialnej [33]. Wskaźnik ten mierzy pole obszaru w przestrzeni celów pokrytego

Tabela 3.1. Charakterystyka ośmiu instancji testowych wykorzystanych w eksperymentach

Instancja	Liczba regionów	Liczba ofiar
<i>Grupa TC-PGI:</i>		
tcB	286	194 005
tcT	450	63 674
tcV	472	59 591
tcW	259	221 848
<i>Grupa TC-SP:</i>		
tcGB	379	843 664
tcGL	599	1 351 466
tcLO	422	894 089
tcSW	558	1 849 239

przez dany zbiór rozwiązań i ograniczonego z góry ustalonym punktem referencyjnym $\mathbf{r}$; im większe, tym lepsza aproksymacja frontu Pareto. Formalnie, dla zbioru $\mathcal{A}$ rozwiązań niezdominowanych [13]:

$$HV(\mathcal{A}, \mathbf{r}) = \lambda\left(\bigcup_{\mathbf{a} \in \mathcal{A}} [\mathbf{a}, \mathbf{r}]\right), \quad (3.1)$$

gdzie $\lambda(\cdot)$ oznacza pole obszaru, a $[\mathbf{a}, \mathbf{r}]$ – prostokąt rozpięty między rozwiązaniem $\mathbf{a}$ a punktem referencyjnym. W niniejszej pracy $M = 2$, więc HV odpowiada polu obszaru zdominowanego przez zbiór rozwiązań. Wskaźnik HV jest zgodny z dominacją Pareto: jeżeli $\mathcal{A}$ dominuje $\mathcal{B}$, to $HV(\mathcal{A}) > HV(\mathcal{B})$ [33]. Jako pojedyncza, skalarna miara jakości nadaje się bezpośrednio do analizy statystycznej.

Normalizacja. Ponieważ f_1 i f_2 przyjmują różne zakresy na różnych instancjach, dla każdej instancji znormalizowano je do zakresu $[0, 1]$ przy użyciu wspólnych granic (minimum i maksimum ze wszystkich archiwów, algorytmów i uruchomień całej pracy), wyznaczonych raz, globalnie. Dzięki temu granice są identyczne dla wszystkich porównywanych algorytmów i eksperymentów, co zapewnia porównywalność wyników zarówno między algorytmami, jak i między tabelami z różnych eksperymentów. Punkt referencyjny ustalono na $\mathbf{r} = (1, 1, 1, 1)$ w przestrzeni znormalizowanej [18, 3]; z jego przesunięcia poza zakres $[0, 1]$ wynika, że maksymalne osiągalne HV wynosi 1,21, przez co w niektórych tabelach wartości dla najlepszych konfiguracji nieznacznie przekraczają 1,0.

3.3. Narzędzia statystyczne

Algorytmy ewolucyjne są losowe – każde uruchomienie z innym ziarnem generatora liczb pseudolosowych może dać inny wynik – więc różnice między nimi ocenia się staty-

stycznie. Poniżej opisano wspólne narzędzia analizy, wykorzystywane w eksperymentach z Sekcji 3.4.

Jednostka replikacji i agregacja. Wszystkie analizy traktują *instancję testową*, a nie pojedyncze uruchomienie, jako jednostkę replikacji: trzydzieści uruchomień tego samego wariantu na tej samej instancji jest ze sobą powiązanych, więc analiza operuje na jednym wyniku na parę (wariant, instancja), zgodnie z zaleceniem dla porównań na wielu zbiorach danych [8]. Trzydzieści uruchomień sprowadza się do *mediany HV* w obrębie instancji – miary odpornej na pojedynczy nietypowy przebieg.

Testy istotności. Do porównania dwóch algorytmów na ośmiu instancjach stosuje się nieparametryczny *test Wilcoxona signed-rank* [16, 8], weryfikujący hipotezę o symetrycznym rozłożeniu różnic δ_i wokół zera przy progu $\alpha = 0,05$. Dla każdej z N instancji wyznacza się różnicę δ_i , nadaje wartościom $|\delta_i|$ rangi R_i i oblicza sumy rang osobno dla różnic dodatnich i ujemnych:

$$W^+ = \sum_{i: \delta_i > 0} R_i, \quad W^- = \sum_{i: \delta_i < 0} R_i, \quad W = \min(W^+, W^-), \quad (3.2)$$

gdzie statystyka W jest podstawą wyznaczenia wartości p ; przy $N = 8$ instancjach wartość tę wyznacza się *testem dokładnym*, bez aproksymacji normalnej. Pomocniczo stosuje się jeszcze prostszy *test znaków* [8], który pomija wielkość różnic i liczy tylko, na ilu z N instancji dany wariant okazał się lepszy. Gdyby oba warianty były jednakowo dobre (hipoteza zerowa), wynik na każdej instancji byłby kwestią przypadku, jednakowo prawdopodobny dla każdego z nich; liczba wygranych miałaby wówczas rozkład dwumianowy $\text{Bin}(N, 0,5)$ – taki sam, jak liczba sukcesów w N niezależnych próbach o prawdopodobieństwie $\frac{1}{2}$. Wynik silnie odbiegający od połowy (np. wygrana na wszystkich ośmiu instancjach) jest przy tej hipotezie mało prawdopodobny, co czyni różnicę istotną; test nie zakłada przy tym symetrii różnic, więc stanowi ostrożniejsze potwierdzenie wyniku testu Wilcoxona.

Gdy jedna analiza obejmuje wiele porównań naraz (rodzinę K testów), rośnie ryzyko, że któryś z nich wypadnie „istotny” wyłącznie przez przypadek. Aby utrzymać łączne prawdopodobieństwo takiego fałszywego wykrycia na poziomie $\alpha = 0,05$, stosuje się korekcję Holma-Bonferroniego [17]: wartości p porządkuje się rosnąco i ocenia coraz łagodniejszym progiem – najmniejszą najsurowiej. Hipotezę o i -tej najmniejszej wartości $p_{(i)}$ odrzuca się, gdy

$$p_{(i)} \leq \frac{\alpha}{K - i + 1}, \quad (3.3)$$

przy czym po pierwszym teście, który tego progu nie przekroczy, pozostałe – o większych wartościach p – również pozostają nieodrzucone.

Wielkość efektu. Wielkość efektu ocenia nieparametryczny wskaźnik A Vargha-Delaneya [29], rekomendowany do porównań algorytmów losowych [1]. Szacuje on prawdopodobieństwo, że losowo wybrane uruchomienie pierwszego wariantu osiągnie

wyższe HV niż uruchomienie drugiego ($A = 0,5$ oznacza brak różnicy). Dla dwóch grup po m_1 i m_2 uruchomień oblicza się go z sumy rang R_1 pierwszej grupy [1]:

$$A = \frac{R_1/m_1 - (m_1 + 1)/2}{m_2}. \quad (3.4)$$

Wskaźnik wyznacza się osobno dla każdej instancji i podsumowuje medianą po ośmiu instancjach, przyjmując progi [29]: $|A - 0,5| < 0,06$ – efekt znikomy, $< 0,14$ – mały, $< 0,21$ – średni, $\geq 0,21$ – duży.

Testy istotności i wskaźnik A działają przy tym na dwóch komplementarnych poziomach: testy – na medianach instancyjnych, oceniając spójność efektu między instancjami, a wskaźnik A – na surowych uruchomieniach w obrębie instancji, mierząc rozdzielanie rozkładów wewnątrz instancji. Jest to rozróżnienie celowe.

3.4. Plan eksperymentów

Wykonano cztery eksperymenty obliczeniowe: inicjalizacyjny, czynnikowy, strojenie hiperparametrów oraz ablację operatorów mutacji. Każdy opisano poniżej łącznie z metodą analizy jego wyników. We wszystkich eksperymentach każdy wariant uruchamiano niezależnie $R = 30$ razy z różnymi ziarnami generatora liczb pseudolosowych, przy budżecie $N_{\text{eval}} = 30\,000$ ewaluacji. Budżet ten ustalono na podstawie krzywych zbieżności HV , które wykazały stabilizację wartości na większości instancji przed jego wyczerpaniem; wyjątek – instancja tcW – omówiono w ograniczeniach (Sekcja 5.4). Tabela 3.2 zestawia parametry stałe wspólne dla wszystkich eksperymentów.

Tabela 3.2. Parametry stałe wszystkich eksperymentów

Parametr	Wartość
Budżet ewaluacji N_{eval}	30 000
Liczba niezależnych uruchomień R	30
Liczba UAV k	3
Liczba instancji testowych	8
Wagi operatorów mutacji w_i (domyślne)	1/6 (jednorodne)

3.4.1. Eksperyment inicjalizacyjny

Pierwszy eksperyment ocenia wpływ strategii inicjalizacji populacji na jakość uzyskiwanych rozwiązań: konkretnie, czy inicjalizacja hybrydowa, wykorzystująca heurystyki konstrukcyjne do utworzenia części osobników startowych, przewyższa inicjalizację czysto losową. Eksperyment przeprowadzono na algorytmie SMS-EMOA, który stanowił punkt wyjścia niniejszej pracy, zanim zasadniczy ciężar analizy przeniósł się na porównanie czterech wariantów architektonicznych (Sekcja 3.4.2). Ponieważ inicjalizacja populacji działa

przed mechanizmami selekcji i architektury, jest od nich niezależna, a jej kod i reprezentacja rozwiązań są identyczne we wszystkich czterech wariantach, wynik uzyskany na SMS-EMOA odnosi się do sposobu tworzenia populacji początkowej jako takiego – wspólnego dla całego zbioru algorytmów.

Uruchomiono dwa warianty SMS-EMOA różniące się wyłącznie sposobem inicjalizacji, przy identycznych pozostałych parametrach:

- **inicjalizacja hybrydowa** – 20% osobników tworzonych metodą najbliższego sąsiada, 10% inicjalizacją klastrową, 10% inicjalizacją priorytetową (regiony z ofiarami) oraz 60% inicjalizacją zrównoważoną losową;
- **inicjalizacja czysto losowa**.

Trzy komponenty heurystyczne wprowadzają do populacji startowej rozwiązania o korzystnej strukturze – trasy krótkie (metoda najbliższego sąsiada), rozdzielone geograficznie (inicjalizacja klastrowa) lub premiujące wczesne odkrywanie ofiar (inicjalizacja priorytetowa) – podczas gdy większościowy komponent losowy (60%) dostarcza różnorodności wymaganej przez operatory krzyżowania i mutacji; pełny opis strategii zawiera Sekcja 2.3. Eksperyment rozstrzyga, czy tak przygotowany „ciepły start” faktycznie podnosi jakość, mimo że zarazem zmniejsza początkową różnorodność populacji względem inicjalizacji czysto losowej.

Łącznie wykonano $2 \times 30 \times 8 = 480$ uruchomień.

Analiza: porównanie strategii inicjalizacji. Dla każdej z ośmiu instancji wyznacza się medianę HV z trzydziestu uruchomień każdego wariantu i oblicza ich różnicę (mediana wariantu hybrydowego minus mediana wariantu losowego). Powstaje osiem różnic instancyjnych, ocenianych testem Wilcozona signed-rank oraz testem znaków; wielkość efektu wyraża wskaźnik A . Porównanie prowadzi się w obrębie tej samej instancji, ponieważ oba warianty inaczej zużywają ciąg liczb losowych na etapie inicjalizacji, więc to samo ziarno nie daje im wspólnego punktu startowego. Analiza obejmuje wyłącznie SMS-EMOA, a nie każdy wariant osobno, więc nie rozstrzyga, czy przewaga inicjalizacji hybrydowej jest jednakowo silna dla każdego z nich; dotyczy to jednak wielkości, a nie kierunku efektu, a przy komponencie wspólnym dla wszystkich wariantów i przy całkowitej przewadze obserwowanej dla SMS-EMOA (Seksja 4.1) odwrócenie tego kierunku dla innego wariantu jest skrajnie nieprawdopodobne. Uzasadnia to przyjęcie inicjalizacji hybrydowej jako domyślnej we wszystkich pozostałych eksperymentach.

3.4.2. Eksperyment czynnikowy

Eksperyment czynnikowy stanowi główną część badań. Zastosowano *plan czynnikowy* 2×2 [22]: dwa czynniki (architektura oraz kryterium selekcji), każdy na dwóch poziomach, dają cztery warianty algorytmu (Tabela 3.3), w których systematycznie zmienia się architekturę (steady-state lub generacyjna) oraz kryterium selekcji osobnika do zastąpienia (wkład w hypervolume lub crowding distance).

Wszystkie cztery warianty uruchomiono przy identycznych wartościach parametrów domyślnych, dobranych zgodnie z zaleceniami dla problemów kombinatorycznych z repre-

Tabela 3.3. Plan czynnikowy 2×2 – cztery warianty algorytmu

	HV-contribution	Crowding distance
Steady-state ($\mu+1$)	SMS-EMOA	SS-NSGA-II
Generacyjna	GEN-SMS-EMOA	NSGA-II

zentacją permutacyjną [10]: rozmiar populacji $\mu = 100$ (zgodnie z ustawieniami referencyjnymi NSGA-II [7]), prawdopodobieństwo krzyżowania $p_c = 0,6$ (typowa wartość dla krzyżowania porządkowego OX) oraz prawdopodobieństwo mutacji $p_m = 1,0$ (jedna perturbacja na potomka). Identyczność hiperparametrów jest kluczowa: jedyną różnicą między komórkami planu jest badany czynnik, dzięki czemu porównanie komórek izoluje sam czynnik. Łącznie wykonano $4 \times 30 \times 8 = 960$ uruchomień.

Analiza: rozłożenie na czynniki projektowe. Analiza rozstrzyga, który z dwóch czynników projektowych algorytmu decyduje o jakości – architektura czy kryterium selekcji – oraz czy oba czynniki oddziałują na siebie. Ponieważ warianty różnią się właśnie tymi dwoma elementami (każdy w dwóch odmianach), ich porównanie rozkłada się na trzy niezależne pytania:

- **czy znaczenie ma architektura** – czy dwa warianty generacyjne różnią się od dwóch steady-state (przy uśrednieniu po kryterium selekcji);
- **czy znaczenie ma kryterium selekcji** – czy dwa warianty z wkładem do hypervolume różnią się od dwóch z crowding distance (przy uśrednieniu po architekturze);
- **czy oba elementy oddziałują na siebie** (interakcja) – czy wpływ jednego zależy od ustawienia drugiego.

Każde z tych pytań ma postać *kontrastu* – ustalonej kombinacji średnich z czterech komórek planu [22]. Formalnie, oznaczając komórki symbolami wariantów z Tabeli 3.3:

- efekt architektury: (SMS-EMOA + SS-NSGA-II) – (GEN-SMS-EMOA + NSGA-II);
- efekt kryterium selekcji: (SMS-EMOA + GEN-SMS-EMOA) – (SS-NSGA-II + NSGA-II);
- interakcja: (SMS-EMOA – SS-NSGA-II) – (GEN-SMS-EMOA – NSGA-II).

Trzy kontrasty są wzajemnie ortogonalne, to jest odpowiadają na trzy rozłączne pytania; istotna interakcja oznaczałaby, że efektów głównych nie wolno interpretować w oderwaniu od siebie. Ponieważ są to zaplanowane z góry, ortogonalne efekty planu czynnikowego [22] – a nie rodzina powtórzonych porównań *post hoc*, jak w eksperymencie strojenia (Seksja 3.4.3) – każdy kontrast ocenia się na poziomie $\alpha = 0,05$ bez korekcji na wielokrotność.

Ponieważ wszystkie cztery warianty uruchomiono na tych samych ośmiu instancjach, dla każdej instancji można obliczyć wartość danego kontrastu z czterech median *HV*, otrzymując osiem liczb – po jednej na instancję – do których stosuje się test Wilcozona signed-rank i test znaków. Interakcja stanowi przy tym kolejną kombinację tego samego rodzaju, badaną identycznie, wobec czego nie wymaga osobnego narzędzia. Wielkość efektu wyraża się jako mediana kontrastu w jednostkach *HV*. Dodatkowo weryfikuje się, czy wynik nie opiera się na pojedynczej instancji: analizę powtarza się osiem razy, za każdym

razem pomijając jedną instancję (analiza *leave-one-out*); utrzymanie kierunku i istotności po usunięciu każdej z nich oznacza, że wniosek nie zależy od żadnej pojedynczej instancji.

3.4.3. Strojenie hiperparametrów

Trzeci eksperyment bada wpływ strojenia hiperparametrów na jakość poszczególnych wariantów, a następnie sprawdza, czy wnioski z eksperymentu czynnikowego utrzymują się, gdy każdy wariant działa w swojej najlepszej konfiguracji – a nie przy wspólnych, przyjętych z góry parametrach domyślnych. Gdyby bowiem ustawienia domyślne faworyzowały któryś z poziomów badanych czynników, obserwowane różnice mogłyby zniknąć po indywidualnym dostrojeniu. Do automatycznego doboru hiperparametrów użyto pakietu *irace* [20].

Narzędzie *irace*. *irace* (*Iterated Racing for Automatic Algorithm Configuration*) to narzędzie w języku R, które dla zadanego algorytmu i zbioru instancji treningowych samodzielnie poszukuje dobrej konfiguracji parametrów. Działa metodą *iterowanego wyścigu* (ang. *iterated racing*), powtarzając trzy kroki:

1. **próbkiwanie** – losuje pulę kandydujących konfiguracji z rozkładu prawdopodobieństwa nad przestrzenią parametrów (na początku jednostajnego);
2. **wyścig** – uruchamia kandydatów na kolejnych instancjach treningowych i po każdej z nich, za pomocą testu statystycznego, odrzuca tych istotnie gorszych od najlepszych; wyścig trwa, dopóki nie wyczerpie przydzielonego budżetu lub nie pozostanie niewielka grupa kandydatów;
3. **zawężanie** – rozkład próbkowania przesuwają w stronę konfiguracji, które przetrwały wyścig (tzw. elit), i zmniejsza swój rozrzut, tak by kolejne konfiguracje losować już w ich pobliżu.

Kroki te powtarzają się aż do wyczerpania łącznego budżetu uruchomień, stopniowo skupiając poszukiwanie wokół najlepszych ustawień; wynikiem jest najlepsza znaleziona konfiguracja.

Konfiguracja strojenia. Każdy z czterech wariantów strojono niezależnie, w identycznych warunkach. Przestrzeń przeszukiwania obejmowała dziewięć hiperparametrów: rozmiar populacji $\mu \in \{50, \dots, 300\}$, prawdopodobieństwa krzyżowania i mutacji $p_c, p_m \in [0, 1]$ oraz sześć wag operatorów mutacji $w_i \in [0, 1]$. Jakość konfiguracji *irace* oceniał wartością $-HV$ zwracaną przez algorytm – ponieważ narzędzie minimalizuje koszt, minimalizacja $-HV$ odpowiada maksymalizacji HV . Budżet strojenia ustalono na 5000 uruchomień algorytmu [20], a zbiór instancji podzielono na sześć treningowych (tcB, tcV, tcW, tcGB, tcGL, tcLO), na których toczył się wyścig, oraz dwie walidacyjne (tcT, tcSW), na których sprawdzano trzy najlepsze znalezione konfiguracje. Podział ten pozwala wykryć konfigurację nadmiernie dopasowaną do instancji treningowych: gdyby strojenie „przeuczyło się” na sześciu instancjach, jego przewaga nie utrzymałaby się na dwóch niewidzianych. Każdy wariant w wyznaczonej dla siebie konfiguracji uruchomiono następnie na wszystkich ośmiu instancjach z 30 ziarnami. Łącznie przeprowadzono cztery kampanie strojenia oraz $4 \times 30 \times 8 = 960$ uruchomień weryfikacyjnych.

Analiza: porównanie dostrojonych wariantów. Analiza przebiega w dwóch krokach. W kroku pierwszym mierzy się wpływ strojenia: weryfikuje się, że poprawia ono każdy wariant, porównując jego wersję dostrojoną z domyślną (jest to porównanie wariantu z samym sobą, a więc metodologicznie czyste). Osiem instancyjnych różnic median HV (dostrojona minus domyślna) ocenia test Wilcozona signed-rank oraz test znaków, a wielkość poprawy – wskaźnik A , przy korekcji Holma-Bonferroniego dla rodziny czterech testów (parowanie po instancji, gdyż strojenie zmienia rozmiar populacji μ). W kroku drugim zestawia się cztery dostrojone warianty i wyłania ten, który w swojej najlepszej konfiguracji osiąga najwyższe HV .

Dodatkowo sprawdza się, czy to uporządkowanie jest zgodne z wynikiem eksperymentu czynnikowego – porównując warianty w parach o tym samym kryterium selekcji (GEN-SMS-EMOA z SMS-EMOA, NSGA-II z SS-NSGA-II). Ma to jednak charakter drugorzędny: skoro każdy wariant niesie własną konfigurację irace (w tym inne μ), różnicy między nimi nie da się w pełni przypisać samej architekturze – jest ona spleciona z różnicą w konfiguracji, więc porównanie nie jest już czystym testem czynnika. Potwierdza ono co najwyżej trwałość praktycznego zalecenia; niezależny pomiar efektu architektury pozostaje w gestii eksperymentu czynnikowego, gdzie czynnik był izolowany.

3.4.4. Ablacja operatorów mutacji

Czwarty eksperyment ustala, które z sześciu operatorów mutacji istotnie przyczyniają się do jakości uzyskiwanych rozwiązań. Strojenie hiperparametrów (Seksja 3.4.3) dobiera wagi operatorów tak, by zmaksymalizować HV , odpowiada jednak na inne pytanie niż postawione tutaj: czy każdy z operatorów jest do osiągnięcia tej jakości potrzebny, czy też część z nich można pominąć bez pogorszenia wyniku. Oba pytania się uzupełniają – wysoka waga nadana operatorowi przez strojenie nie przesądza, że jego obecność faktycznie podnosi HV [12]. Do rozstrzygnięcia drugiego z nich zastosowano *analizę ablacyjną* w ujęciu Fawcetta i Hoosa [12] – metodę badającą wkład poszczególnych składników algorytmu poprzez ich kolejne usuwanie i obserwację zmiany jakości.

Konfiguracja bazowa i jej zamrożenie. Punktem wyjścia jest najlepszy wariant wyłoniony w strojeniu – GEN-SMS-EMOA w dostrojonej konfiguracji: liczebność populacji $\mu = 90$, prawdopodobieństwa krzyżowania i mutacji p_c, p_m oraz wektor sześciu wag operatorów mutacji (wartości liczbowe zebrano w Tabeli 4.4). Konfigurację tę zamrożono: w toku całej ablacji nie zmieniano żadnego jej parametru poza usuwanym operatorem i nie przeprowadzano ponownego strojenia. Zamrożenie jest tu istotne. Gdyby dla każdego zmniejszonego zestawu wagi dobierać od nowa narzędziem irace, zmiana HV łączyłaby dwa efekty naraz – usunięcie operatora i losowy szum samego strojenia – których nie dałoby się potem rozdzielić. Zamrożenie konfiguracji izoluje sam efekt usunięcia; ceną jest węższa interpretacja wyniku (por. ograniczenia na końcu sekcji).

Operacja usunięcia operatora. Operatory mutacji wybierane są losowo, z prawdopodobieństwem proporcjonalnym do ich wag $\mathbf{w} = (w_o)_{o \in \mathcal{O}}$ nad zbiorem sześciu operatorów $\mathcal{O}$; mechanizm ten opisano w Sekcji 2.5.1. Usunięcie operatora odpowiada nadaniu mu wagi 0, co wyłącza go z losowania, oraz *renormalizacji* wag pozostałych, tak aby nadal

sumowały się do jedności. Dla zestawu aktywnych operatorów $\mathcal{S} \subseteq \mathcal{O}$ wagi po usunięciu wynoszą

$$w'_o = \begin{cases} \frac{w_o}{\sum_{o' \in \mathcal{S}} w_{o'}} & \text{gdy } o \in \mathcal{S}, \\ 0 & \text{gdy } o \notin \mathcal{S}. \end{cases} \quad (3.5)$$

Dzięki renormalizacji usunięcie operatora nie zmniejsza łącznej częstości mutacji – waga usuniętego operatora rozkłada się proporcjonalnie na pozostałe, a te zachowują swoje wzajemne proporcje. Poza wektorem wag żaden inny element algorytmu nie ulega zmianie.

Analiza: ocena ważności operatorów. Zastosowano *zachłanną eliminację wsteczną* (Algorytm 5) – standardowe podejście ablacyjne [12], którego celem jest nie tyle znalezienie najlepszego z $2^6 - 1 = 63$ podzbiorów operatorów, ile wyznaczenie *ścieżki ważności*: kolejności, w jakiej operatory stają się zbędne. Procedura wychodzi od pełnego zestawu sześciu operatorów i na każdym kroku usuwa ten operator, którego usunięcie najmniej obniża HV (a jeśli któregoś z usunąć HV podnosi – ten, który podnosi je najbardziej). Wybrany operator zostaje trwale odrzucony, a procedura powtarza się na zredukowanym zestawie, aż pozostanie jeden operator. Powstaje w ten sposób *ścieżka ablacji* – uporządkowany ciąg konfiguracji od sześciu do jednego operatora, z wartością HV przypisaną każdemu poziomowi.

Każdą konfigurację na ścieżce oceniano dokładnie tym samym protokołem, co pozostałe eksperymenty: na ośmiu instancjach testowych, po 30 uruchomieniach z ustalonymi ziarnami, przy budżecie 30 000 ewaluacji funkcji celu, a HV liczono na wspólnych granicach normalizacji (Sekcja 3.2). Wynik pojedynczej konfiguracji $\mathcal{S}$ zagregowano w jedną liczbę identycznie jak w pozostałych analizach – medianą po ziarnach w obrębie instancji, a następnie średnią po instancjach:

$$\overline{HV}(\mathcal{S}) = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \text{median}_{s=1, \dots, 30} HV_{i,s}(\mathcal{S}), \quad (3.6)$$

gdzie $\mathcal{I}$ jest zbiorem ośmiu instancji, a $HV_{i,s}(\mathcal{S})$ – znormalizowanym HV uzyskanym na instancji i z ziarnem s .

Ścieżka niesie dwie uzupełniające się informacje. Po pierwsze, jej pierwszy krok (przejście od sześciu do pięciu operatorów) ocenia usunięcie każdego z sześciu operatorów z pełnej konfiguracji, co daje ważność każdego operatora z osobna, mierzoną spadkiem HV względem kompletnego zestawu (raportowanego w Sekcji 4.4). Po drugie, przebieg HV wzdłuż całej ścieżki wskazuje najlepszy napotkany podzbiór operatorów (jej szczyt) oraz moment, w którym dalsze usuwanie zaczyna wyraźnie szkodzić, rozdzielając operatory *niezbędne* (ich usunięcie mocno obniża HV) od *zbędnych* (ich usunięcie nie pogarsza wyniku lub go poprawia). Cała procedura wymaga oceny 21 konfiguracji (jednej pełnej oraz $6 + 5 + 4 + 3 + 2$ kandydatów rozważanych na pięciu krokach), co przy ośmiu instancjach i 30 ziarnach daje 5 040 uruchomień algorytmu.

Ograniczenia metody. Przyjęte podejście niesie trzy ograniczenia, które należy jawnie zaznaczyć. Po pierwsze, mierzy ono ważność operatora wewnątrz jednej, zamrożonej konfiguracji, a nie najlepszy możliwy algorytm o zredukowanej liczbie operatorów; wag

Algorithm 5 Zachłanna eliminacja wsteczna operatorów mutacji

```

1:  $\mathcal{S} \leftarrow \mathcal{O}$  ▷ pełny zestaw
2: zapisz na ścieżce parę  $(\mathcal{S}, \overline{HV}(\mathcal{S}))$ 
3: while  $|\mathcal{S}| > 1$  do
4:   for all  $o \in \mathcal{S}$  do
5:      $\mathcal{S}_o \leftarrow \mathcal{S} \setminus \{o\}$ 
6:     wyznacz  $\overline{HV}(\mathcal{S}_o)$  ▷ wg (3.5) i (3.6)
7:   end for
8:    $o^* \leftarrow \arg \max_{o \in \mathcal{S}} \overline{HV}(\mathcal{S} \setminus \{o\})$  ▷ najmniejsza strata  $HV$ 
9:    $\mathcal{S} \leftarrow \mathcal{S} \setminus \{o^*\}$ 
10:  zapisz na ścieżce parę  $(\mathcal{S}, \overline{HV}(\mathcal{S}))$  oraz usunięty  $o^*$ 
11: end while
12: return ścieżka ablacji

```

nie strojono ponownie dla mniejszych zestawów, gdyż – jak wyżej – powtórne strojenie wprowadziłoby z powrotem zmienność narzędzia irace. Po drugie, skutek renormalizacji (równanie 3.5) mierzony jest *efekt netto* usunięcia operatora – łącznie z przeniesieniem jego wagi na operatory pozostałe – a nie wpływ operatora rozpatrywanego w całkowitej izolacji. Po trzecie, eliminacja zachłanna sprawdza tylko jedną ścieżkę usuwania i może pominąć globalnie najlepszy zestaw operatorów [12]; analizę prowadzono wyłącznie w kierunku od pełnego zestawu w dół.

Rozdział 4

Wyniki

Rozdział przedstawia wyniki czterech eksperymentów opisanych w Sekcji 3.4, w tej samej kolejności: zasadność inicjalizacji hybrydowej (Sekcja 4.1), wpływ architektury i kryterium selekcji (Sekcja 4.2), strojenie hiperparametrów (Sekcja 4.3) oraz ablację operatorów mutacji (Sekcja 4.4). Rozdział zamyka dyskusja wyników (Sekcja 4.5).

4.1. Zasadność inicjalizacji hybrydowej

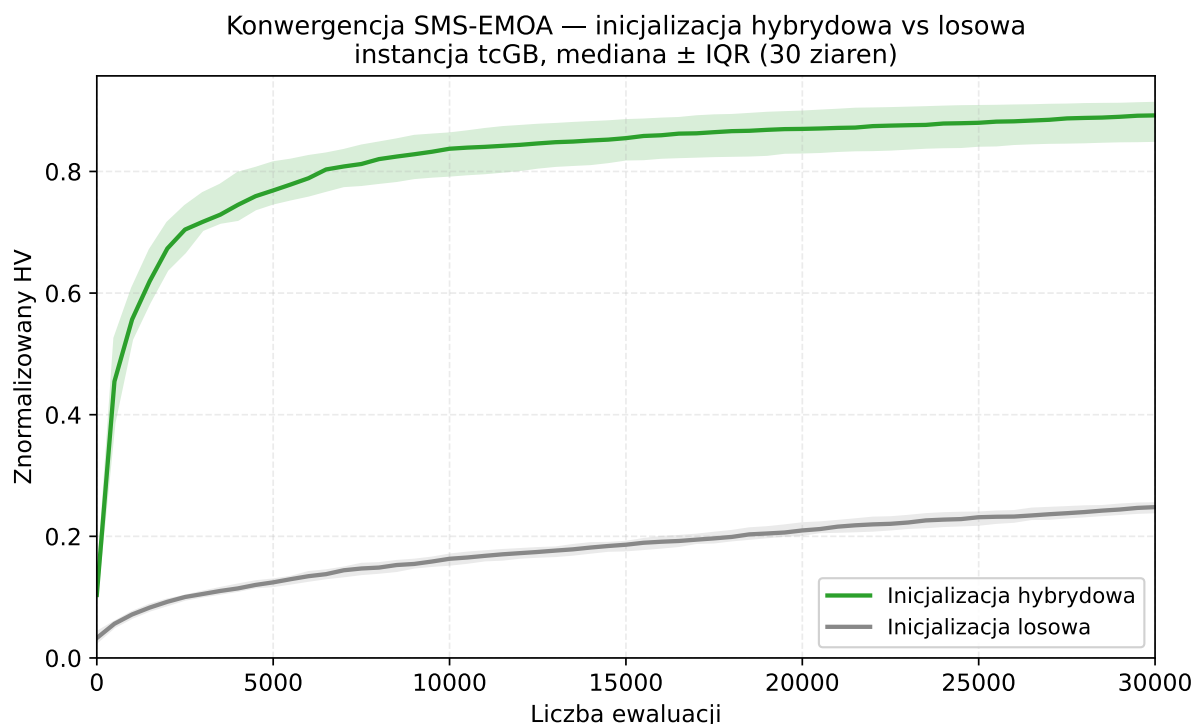
Pierwsza analiza porównuje algorytm SMS-EMOA z inicjalizacją hybrydową i czysto losową na ośmiu instancjach (Sekcja 3.4.1). Tabela 4.1 zestawia mediany HV obu wariantów oraz wskaźnik wielkości efektu A .

Tabela 4.1. Mediany HV inicjalizacji hybrydowej i czysto losowej (SMS-EMOA, 30 uruchomień na instancję) oraz wielkość efektu A

Instancja	HV hybrydowa	HV losowa	A
tcB	0,950	0,344	1,000
tcT	0,932	0,192	1,000
tcV	0,960	0,208	1,000
tcW	0,664	0,399	1,000
tcGB	0,892	0,248	1,000
tcGL	0,868	0,149	1,000
tcLO	0,965	0,250	1,000
tcSW	0,983	0,181	1,000

Inicjalizacja hybrydowa osiąga wyższe HV na każdej z ośmiu instancji, a przewaga jest przy tym ogromna: wskaźnik $A = 1,000$ na każdej instancji oznacza *całkowitą separację próbek* – w każdej z $30 \times 30 = 900$ par uruchomień wariant hybrydowy osiąga wyższe HV niż losowy, bez jednego wyjątku. Różnice median są odpowiednio duże, na większości instancji wielokrotne (np. tcSW: 0,983 wobec 0,181; tcLO: 0,965 wobec 0,250). Przy tak jednoznacznym rozdzieleniu wyników testy istotności pełnią jedynie rolę formalnego potwierdzenia: zarówno test Wilcoxona signed-rank, jak i test znaków osiągają najniższą wartość p możliwą przy $N = 8$ instancjach, niezależnie od tego, czy uruchomienia zwinąć medianą, czy średnią.

Skalę różnicy dobrze ilustruje przebieg zbieżności (Rysunek 4.1): wariant losowy do końca budżetu 30 000 ewaluacji osiąga poziom HV , który wariant hybrydowy przekracza już we wczesnej fazie przebiegu – różnica nie jest więc kwestią szybkości zbieżności, lecz jakości rozwiązań. Wariant losowy startuje z tras o przypadkowej strukturze i w ramach budżetu nie nadrabia dystansu do populacji zaszczerpionej heurystykami konstrukcyjnymi (Sekcja 3.4.1). Efekt tej skali, dotyczący komponentu wspólnego dla wszystkich czterech wariantów (Sekcja 3.4.1), uzasadnia przyjęcie inicjalizacji hybrydowej jako domyślnej w pozostałych eksperymentach.



Rysunek 4.1. Krzywe zbieżności znormalizowanego HV dla inicjalizacji hybrydowej i czysto losowej (SMS-EMOA) na instancji tcGB. Każda krzywa to mediana z 30 uruchomień, zacieniony obszar – rozstęp międzykwartyłowy (IQR). Pasma nie nakładają się na żadnym etapie przebiegu.

4.2. Wpływ architektury i kryterium selekcji

Druga analiza, na danych eksperymentu czynnikowego (Sekcja 3.4.2), rozstrzyga, który z dwóch elementów konstrukcji algorytmu – architektura czy kryterium selekcji – decyduje o jakości i czy elementy te oddziałują na siebie. Tabela 4.2 zestawia mediany HV czterech wariantów na ośmiu instancjach, a wyniki trzech kontrastów podsumowuje Tabela 4.3.

Architektura decyduje o jakości. Architektura generacyjna osiąga wyższe HV od steady-state na siedmiu z ośmiu instancji (jedynym wyjątkiem jest tcLO, gdzie wszystkie warianty osiągają zbliżone, wysokie wartości). Różnica jest istotna statystycznie (Tabela 4.3) i niezależna od sposobu agregacji median. Ostrożniejszy test znaków nie prze-

Tabela 4.2. Mediany HV czterech wariantów algorytmu na ośmiu instancjach (30 uruchomień na instancję, parametry domyślne). Warianty pogrupowano według architektury.

Inst.	Steady-state		Generacyjna	
	SMS-EMOA	SS-NSGA-II	GEN-SMS-EMOA	NSGA-II
tcB	0,9499	0,9570	0,9680	0,9702
tcT	0,9320	0,9272	0,9373	0,9372
tcV	0,9597	0,9530	0,9681	0,9577
tcW	0,6642	0,6831	0,8271	0,8394
tcGB	0,8922	0,8991	0,9110	0,9068
tcGL	0,8681	0,8594	0,8654	0,8651
tcLO	0,9646	0,9755	0,9696	0,9680
tcSW	0,9831	0,9795	0,9840	0,9793

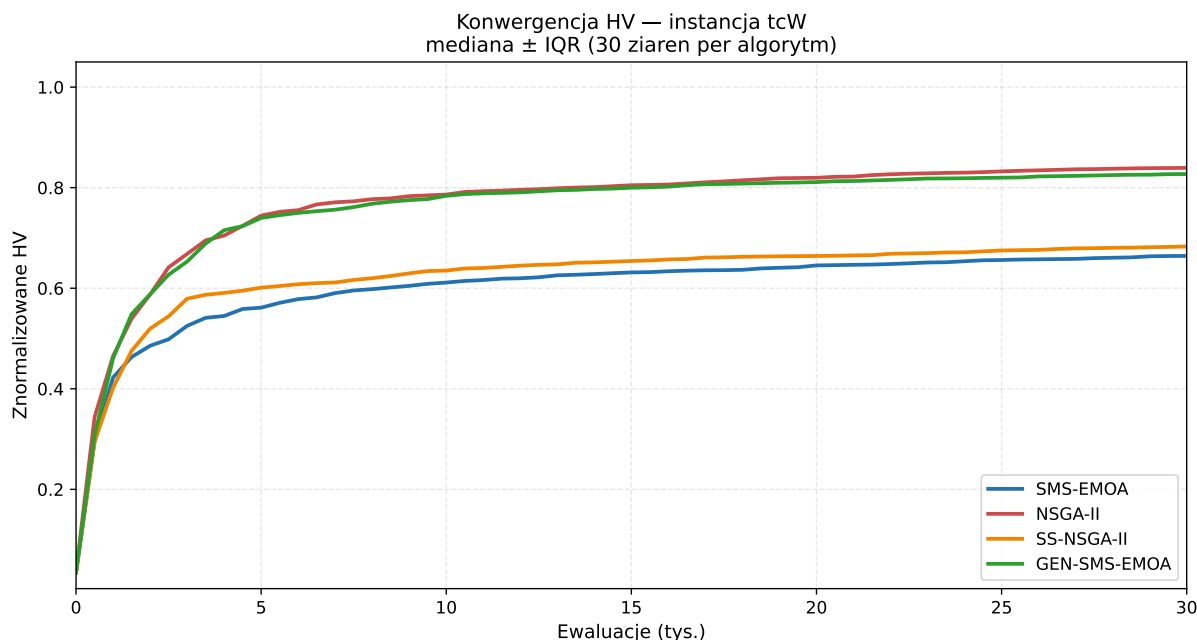
Tabela 4.3. Wynik trzech ortogonalnych kontrastów. *Wynik* – słowny werdykt („7 z 8” = architektura generacyjna lepsza na siedmiu z ośmiu instancji; „remis 4:4” = po cztery instancje na każde kryterium selekcji). *Wielkość* – mediana kontrastu w jednostkach HV . *p* – test Wilcoxon signed-rank na ośmiu medianach instancyjnych.

Efekt	Wynik	Typowa różnica [HV]	p
Architektura	generacyjna (7 z 8)	0,007	0,023
Kryterium selekcji	remis (4:4)	< 0,001	0,844
Interakcja	brak	0,004	0,250

kracza progu istotności przy $N = 8$ (siła 7/8), lecz spójny kierunek na niemal wszystkich instancjach czyni przewagę jednoznaczną co do znaku.

Wielkość tej przewagi jest jednak umiarkowana i silnie zależy od instancji. Na sześciu z ośmiu instancji różnica median wynosi zaledwie 0,001–0,016 w skali HV , natomiast na najtrudniejszej instancji tcW osiąga aż 0,16 (warianty generacyjne $HV \approx 0,83$ –0,84 wobec $\approx 0,66$ –0,68 dla steady-state). Przewaga architektury generacyjnej jest zatem spójna kierunkowo, lecz co do wielkości skoncentrowana tam, gdzie instancja faktycznie różnicuje warianty; na łatwiejszych instancjach, na których wszystkie warianty osiągają zbliżone, wysokie wartości HV , efekt zanika. Dobrze widać to na krzywych zbieżności na tcW (Rysunek 4.2): para generacyjna utrzymuje przewagę nad parą steady-state przez cały przebieg, a warianty różniące się jedynie kryterium selekcji niemal się pokrywają.

Ponieważ pojedyncza instancja (tcW) wnosi największą różnicę bezwzględną, sprawdzono, czy to nie ona samodzielnie napędza wynik. Test rangowy jest na to odporny z natury – wykorzystuje kolejność, a nie surową wielkość – a analiza leave-one-out to potwierdza liczbowo: po usunięciu dowolnej z ośmiu instancji – w tym samej tcW – kontrast pozostaje istotny (wartość p w przedziale 0,016–0,047), a jego mediana nie zmienia znaku



Rysunek 4.2. Krzywe zbieżności znormalizowanego HV czterech wariantów na najtrudniejszej instancji tcW (mediana z 30 uruchomień, zacieniony obszar – IQR). Dwa warianty generacyjne (NSGA-II, GEN-SMS-EMOA) odrywają się w górę od dwóch steady-state (SMS-EMOA, SS-NSGA-II) – linie grupują się według architektury. W obrębie każdej pary warianty różniące się jedynie kryterium selekcji (wkład do hypervolume vs. crowding distance) niemal się pokrywają, co ilustruje brak wpływu tego kryterium.

(utrzymuje się na poziomie $+0,007$ w skali HV). Przewaga architektury generacyjnej nie jest więc artefaktem pojedynczej instancji.

Nie wykryto wpływu kryterium selekcji. Zmiana kryterium selekcji osobnika do zastąpienia – z wkładu w hypervolume na crowding distance – nie różnicuje wyników. Kontrast jest nieistotny (Tabela 4.3), mediana efektu praktycznie zerowa, a kierunek różnicy niespójny: na czterech instancjach lepsze okazuje się jedno kryterium, na czterech drugie. Dla rozważanego problemu nie wykryto mierzalnego wpływu wyboru między tymi dwoma kryteriami selekcji środowiskowej na jakość rozwiązań.

Czynniki nie oddziałują na siebie. Kontrast interakcji jest nieistotny (Tabela 4.3), co oznacza, że wpływ architektury nie zależy od wybranego kryterium selekcji (i odwrotnie). Oba efekty główne można zatem interpretować rozłącznie: przewaga architektury generacyjnej utrzymuje się niezależnie od kryterium selekcji, a brak wpływu kryterium – niezależnie od architektury.

Podsumowanie. Spośród dwóch badanych elementów konstrukcji algorytmu o jakości decyduje wyłącznie architektura: warianty generacyjne przewyższają steady-state w sposób spójny i odporny na usunięcie dowolnej instancji, choć wyraźny co do wielkości głównie na instancjach trudnych. Kryterium selekcji środowiskowej pozostaje bez wpływu, a oba

czynniki nie oddziałują na siebie. Najlepsze wyniki osiągają dwa warianty generacyjne – GEN-SMS-EMOA i NSGA-II.

4.3. Strojenie hiperparametrów

Trzecia analiza ma cel praktyczny: dostroić każdy wariant do jego najlepszej konfiguracji (narzędziem irace, Sekcja 3.4.3) i sprawdzić, który wtedy wypada najlepiej. Najpierw potwierdzamy, że strojenie faktycznie poprawia każdy wariant, następnie zestawiamy dostrojone warianty i wyłaniamy najlepszy. Drugorzędnie sprawdzamy, czy uporządkowanie wciąż odtwarza przewagę architektury generacyjnej – z zastrzeżeniem, że przy różnych konfiguracjach nie jest to już czyste porównanie samego czynnika.

4.3.1. Dostrojone konfiguracje

Tabela 4.4 zestawia konfiguracje wyznaczone przez irace osobno dla każdego wariantu. Mimo że strojenie prowadzono niezależnie, widać kilka powtarzalnych wzorców. Operator *invert* otrzymał wysoką wagę we wszystkich czterech wariantach (najwyższą w trzech z nich), a podobnie wysoko irace wyceniło *cross-route* – są to dokładnie te dwa operatory, które ablacja wskaże później jako niezbędne (Sekcja 4.4). Prawdopodobieństwo krzyżowania ustaliło się we wszystkich wariantach znacznie poniżej wartości domyślnej 0,6 (zakres 0,20–0,33), a rozmiar populacji pozostał w pobliżu domyślnych 100 osobników (72–108) – strojenie nie sięgnęło ku górnej granicy zakresu (300). Pozostałe wagi operatorów różnią się między wariantami wyraźniej i nie ustaliły się na wspólnym poziomie.

Tabela 4.4. Konfiguracje hiperparametrów wyznaczone przez irace niezależnie dla każdego z czterech wariantów.

Parametr	SMS-EMOA	NSGA-II	SS-NSGA-II	GEN-SMS-EMOA
μ (populacja)	108	72	106	90
p_c	0,279	0,200	0,328	0,293
p_m	0,939	0,996	0,757	0,934
w_{invert}	0,821	0,872	0,765	0,916
$w_{\text{or-opt}}$	0,242	0,410	0,642	0,707
w_{transfer}	0,359	0,268	0,132	0,275
w_{swap}	0,042	0,051	0,380	0,158
$w_{\text{cross-route}}$	0,748	0,705	0,926	0,638
w_{exchange}	0,051	0,612	0,328	0,025

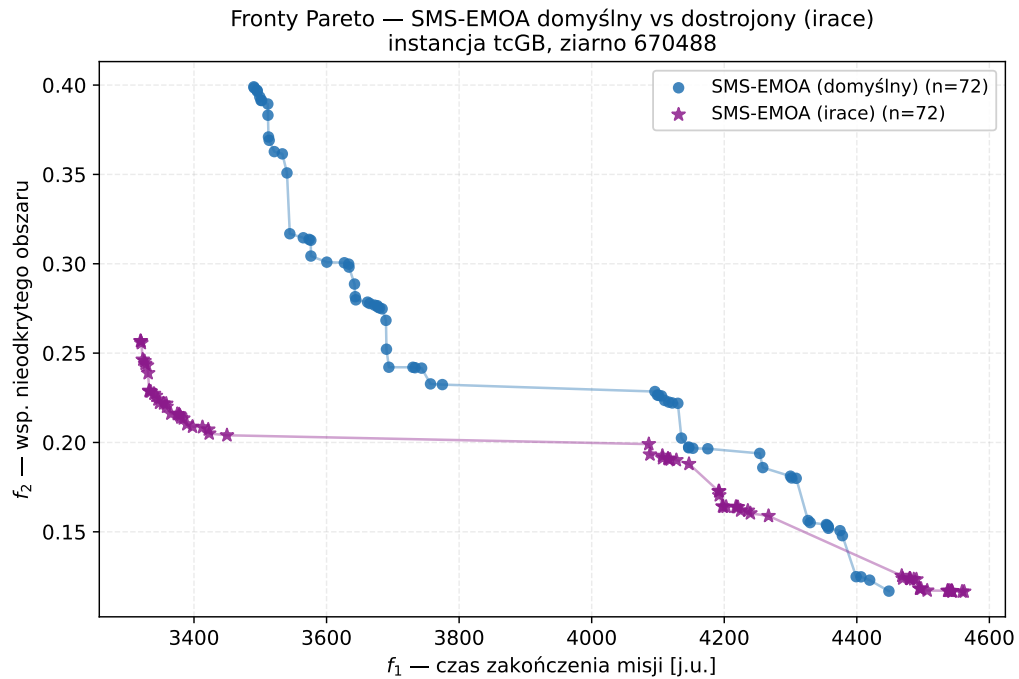
4.3.2. Który wariant wypada najlepiej

Tabela 4.5 zestawia cztery dostrojone warianty, uporządkowane według osiągniętego *HV*, wraz z poprawą względem wersji domyślnej – dla kontroli, że każdy został faktycznie zoptymalizowany.

Tabela 4.5. Cztery dostrojone warianty, uporządkowane według HV . HV (dostrojony) – średnia po ośmiu instancjach; $Poprawa$ – mediana różnicy względem wersji domyślnej (dostrojony minus domyślny); A – prawdopodobieństwo, że dostrojony przebieg da wyższe HV niż domyślny.

Wariant	HV (dostrojony)	$Poprawa [HV]$	A
GEN-SMS-EMOA	0,952	+0,025	0,79
NSGA-II	0,949	+0,025	0,75
SMS-EMOA	0,935	+0,023	0,72
SS-NSGA-II	0,912	+0,007	0,61

Strojenie poprawia każdy wariant. Trzy z nich – GEN-SMS-EMOA, NSGA-II i SMS-EMOA – zyskują wyraźnie, z dużym efektem ($A \approx 0,72$ – $0,79$; w ok. 72–79% par uruchomień konfiguracja dostrojona daje wyższe HV) – efekt ten ilustruje Rysunek 4.3, na którym dostrojony front SMS-EMOA dominuje front domyślny. SS-NSGA-II zyskuje najmniej (efekt mały): jego dostrojona konfiguracja poprawia siedem instancji, lecz regreduje na najtrudniejszej tcW , co netto daje jedynie niewielki przyrost. Przy $N = 8$ i korekcji Holma formalna istotność tych poprawek jest graniczna, lecz rozstrzygają duże i spójne wielkości efektu, a nie sama wartość p , która przy tak małej liczbie instancji szybko traci moc.



Rysunek 4.3. Reprezentatywny przykład wpływu strojenia – fronty Pareto SMS-EMOA w konfiguracji domyślnej i dostrojonej (instancja tcGB, pojedyncze uruchomienie). Osie to oba kryteria: czas zakończenia misji f_1 oraz współczynnik nieodkrytego obszaru f_2 (mniej znaczy lepiej dla obu). Front dostrojony (gwiazdki) dominuje domyślny – przy tym samym czasie misji pokrywa większy obszar, a przy tym samym pokryciu kończy misję szybciej.

Po dostrojeniu na czele stają dwa warianty generacyjne – GEN-SMS-EMOA (0,952) i NSGA-II (0,949) – praktycznie nierozróżnialne między sobą (różnica nieistotna). Oba wyprzedzają warianty steady-state (SMS-EMOA 0,935, SS-NSGA-II 0,912). Praktyczne zalecenie jest więc jednoznaczne: gdy każdy algorytm działa w swojej najlepszej konfiguracji, najlepsze wyniki daje architektura generacyjna, a wybór kryterium selekcji pozostaje bez znaczenia – dwa czołowe warianty różnią się właśnie tylko nim i są równorzędne.

Drugorzędnie warto odnotować, że powyższe uporządkowanie (Tabela 4.5) wciąż odzwierciedla przewagę architektury generacyjnej – i to instancja po instancji. Porównując warianty w parach o tym samym kryterium selekcji, tak by różniła się w nich tylko architektura (Tabela 4.6), widać, że w obu parach architektura generacyjna wygrywa na wszystkich ośmiu instancjach, a różnica jest istotna po korekcji Holma.

Tabela 4.6. Porównanie wariantów generacyjnych ze steady-state po dostrojeniu, w parach o tym samym kryterium selekcji. *A* – prawdopodobieństwo, że przebieg generacyjny da wyższe *HV* niż steady-state; obie różnice istotne po korekcji Holma.

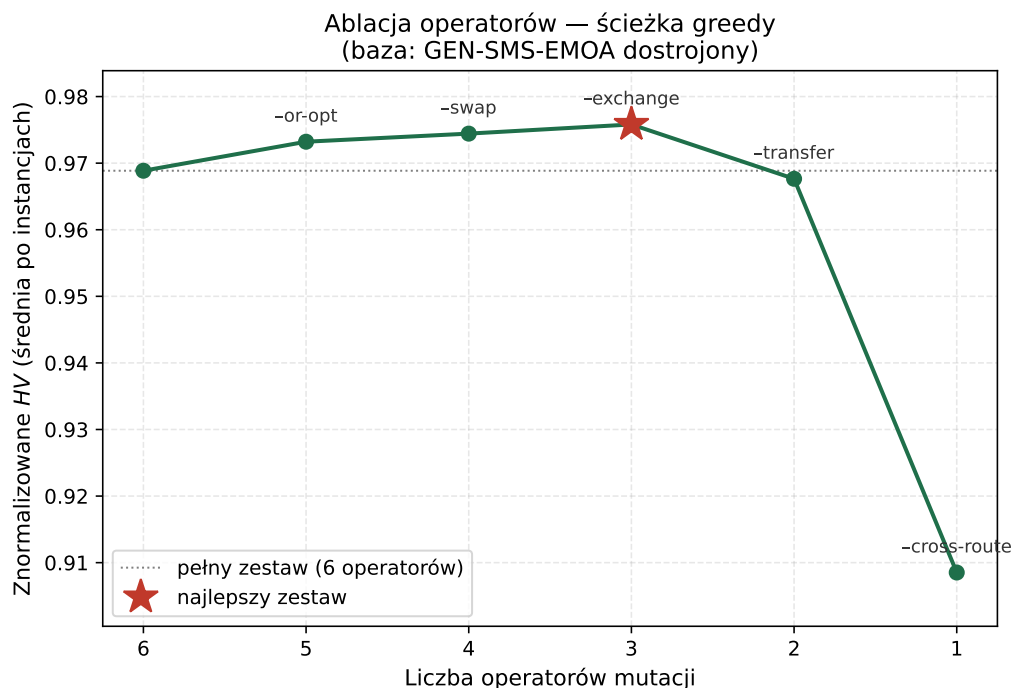
Para (to samo kryterium)	Wynik	Typowa różnica [<i>HV</i>]	<i>A</i>
GEN-SMS-EMOA vs SMS-EMOA (<i>HV</i>)	generacyjna (8 z 8)	+0,010	0,63
NSGA-II vs SS-NSGA-II (crowding)	generacyjna (8 z 8)	+0,016	0,71

Wynik ten wymaga jednak ostrożnej interpretacji. Inaczej niż w eksperymencie czynnikowym (Sekcja 4.2), gdzie wszystkie warianty miały identyczne parametry, tutaj każdy niesie własną konfigurację irace (m.in. inny rozmiar populacji, Tabela 4.4) – różnicy między wariantami nie można więc w pełni oddzielić od różnic w konfiguracji. Analiza ta potwierdza zatem *odporność praktycznego zalecenia* („po dostrojeniu wybieraj architekturę generacyjną”), a nie stanowi niezależnego pomiaru efektu architektury; ten pozostaje w gestii eksperymentu czynnikowego, gdzie czynnik był izolowany.

4.4. Ablacja operatorów mutacji

Ostatni eksperyment ocenia, które z sześciu operatorów mutacji faktycznie przyczyniają się do jakości rozwiązań, usuwając je pojedynczo z najlepszego dostrojonego wariantu (GEN-SMS-EMOA) metodą zachłannej eliminacji wstecznej (Sekcja 3.4.4). Analiza dostarcza dwóch uzupełniających się obrazów: *ścieżki ablacji*, czyli przebiegu *HV* w miarę usuwania kolejnych operatorów (Rysunek 4.4), oraz ważności każdego operatora z osobna, mierzonej spadkiem *HV* po jego usunięciu z pełnego zestawu (Tabela 4.7).

Ścieżka ablacji. Pełny zestaw sześciu operatorów osiąga $\overline{HV} = 0,969$. Trzy pierwsze usunięcia nie tylko nie szkodzą, lecz nieznacznie podnoszą wynik: kolejno *or-opt* (do 0,973), *swap* (do 0,974) i *exchange* (do 0,976). Szczyt ścieżki przypada zatem na zestaw trzech operatorów – *invert*, *cross-route* i *transfer* – z $\overline{HV} = 0,976$. Dalsze usuwanie wyraźnie już szkodzi: odrzucenie *transfer* obniża wynik do 0,968, a pozostawienie samego *invert* – aż do 0,909. Kształt ścieżki jest więc charakterystyczny: płaski, lekko rosnący odcinek



Rysunek 4.4. Ścieżka ablacji operatorów mutacji na dostrojonym wariancie GEN-SMS-EMOA: znormalizowane HV (średnia po ośmiu instancjach) w miarę usuwania operatorów po jednym (od sześciu do jednego). Przy każdym punkcie podano usunięty operator; gwiazdka – najlepszy zestaw (trzy operatory).

przy usuwaniu operatorów zbędnych, wyraźny szczyt, a następnie załamanie w chwili, gdy zaczynają znikać operatory niosące jakość.

Które operatory są niezbędne. Ważność poszczególnych operatorów, mierzona spadkiem HV po usunięciu z pełnego zestawu, zestawia Tabela 4.7. Dwa operatory wyróżniają się jako niezbędne: usunięcie *invert* obniża HV najsilniej ($-0,046$), a tuż za nim plasuje się *cross-route* ($-0,026$). Oba dokonują większych, strukturalnych zmian, przestawiając całe fragmenty tras, przy czym *invert* działa wewnątrz jednej trasy, a *cross-route* między trasami – niezbędny okazuje się więc po jednym operaterze z każdego typu. Należy przy tym zaznaczyć, że ablacja wskazuje na te konkretne operatory w ramach badanego zestawu, lecz nie rozstrzyga, czy nie zastąpiłby ich inny operator tego samego typu (inna perturbacja wewnątrz- lub między-trasowa). Ich rola uwidacznia się najdobitniej na końcu ścieżki: konfiguracja z samym *invert* osiąga zaledwie $\overline{HV} = 0,909$, a spadek ten napędza przede wszystkim najtrudniejsza instancja tcW, na której HV załamuje się do 0,63 (wobec wartości bliskich 1,0 na instancjach łatwych) – to właśnie *cross-route* pozwala sobie z nią poradzić. Pozostałe cztery operatory można natomiast usuwać niemal bez kosztu: ich indywidualny wpływ na HV nie przekracza 0,009, a w przypadku *or-opt* jest wręcz dodatni.

Trzy operatory wystarczają. Choć szczyt ścieżki (trzy operatory) leży nieco wyżej niż pełny zestaw (0,976 wobec 0,969), różnica ta jest niewielka i statystycznie nieistotna – ani

Tabela 4.7. Wpływ usunięcia każdego operatora z pełnego (dostrojonego) zestawu na *HV*. Wartość ujemna – usunięcie szkodzi (operator ważny); dodatnia – usunięcie pomaga.

Operator	ΔHV przy usunięciu	Wynik
invert	−0,046	niezbędny
cross-route	−0,026	niezbędny
transfer	−0,009	ważny
swap	−0,006	zbędny
exchange	−0,005	zbędny
or-opt	+0,004	zbędny (usunięcie pomaga)

test Wilcoxona ($p = 0,31$), ani test znaków ($p = 0,73$) nie pozwalają odrzucić hipotezy o braku różnicy między zestawem trzech a pełnym. Wniosek należy zatem formułować ostrożnie: *trzy operatory wystarczają*, aby dorównać pełnemu zestawowi – pozostałe trzy są zbędne – a nie „trzy są istotnie lepsze od sześciu”. W praktyce oznacza to, że połowę operatorów mutacji można pominąć bez mierzalnej utraty jakości, upraszczając algorytm.

Paradoks operatora or-opt. Na szczególną uwagę zasługuje operator *or-opt*. Strojenie przypisało mu drugą co do wielkości wagę (0,71; Tabela 4.4), a mimo to jego usunięcie poprawia *HV*. Pokazuje to, że waga nadana operatorowi przez strojenie nie jest miarą jego przydatności – ablacja odpowiada na inne pytanie niż irace (które operatory faktycznie się liczą) i jest analizą uzupełniającą wobec strojenia [12], a nie jego powtórzeniem. Cały wynik należy odczytywać jako ważność operatora wewnątrz dostrojonej konfiguracji, z zastrzeżeniami z Sekcji 3.4.4 – w szczególności jako *efekt netto* usunięcia, mierzony łącznie z renormalizacją wag pozostałych operatorów.

4.5. Dyskusja

Przeprowadzone eksperymenty układają się w spójny obraz. Inicjalizacja hybrydowa okazała się warunkiem koniecznym sensownej jakości – jej efekt (całkowita separacja próbek względem inicjalizacji losowej) jest największym obserwowanym w całej pracy. Spośród dwóch badanych czynników projektowych o jakości decyduje wyłącznie architektura: warianty generacyjne przewyższają steady-state, podczas gdy kryterium selekcji środowiskowej (wkład do hypervolume vs. crowding distance) pozostaje bez wpływu – w praktyce oznacza to, że oba kryteria są równorzędne. Strojenie hiperparametrów poprawia każdy wariant, lecz nie odwraca tej hierarchii: po dostrojeniu najlepsze pozostają dwa warianty generacyjne. Ablacja operatorów pokazała ponadto, że za jakość odpowiada wąski podzbiór operatorów mutacji: dwa z sześciu są niezbędne, a trzy wystarczają.

Dlaczego architektura generacyjna wygrywa. Ponieważ wszystkie cztery warianty dzielą identyczną reprezentację, funkcje celu, inicjalizację oraz komplet operatorów,

a selekcję rodziców ujednolicono, jedyną różnicą między architekturami pozostaje schemat zastępowania. Możliwe wyjaśnienie obserwowanej przewagi wiąże się z różnorodnością populacji. Schemat steady-state stosuje silniejszą presję selekcyjną i szybciej „zawłaszcza” populację najlepszymi osobnikami niż schemat generacyjny [15]; przyspiesza to zbieżność, lecz kosztem szybszej utraty różnorodności. Na rozległej, silnie wielomodalnej przestrzeni permutacyjnej (rzędu $n!$) rozważanego problemu ta zachłanność sprzyja przedwczesnej zbieżności, podczas gdy generacyjne zastępowanie całą pulą potomków dłużej utrzymuje różnorodność. Hipoteza ta jest spójna z obserwacją, że przewaga architektury generacyjnej koncentruje się na najtrudniejszej instancji tcW, gdzie koszt przedwczesnej zbieżności jest największy. Pozostaje ona jednak hipotezą – jej bezpośrednia weryfikacja wymagałaby pomiaru różnorodności populacji w trakcie działania algorytmu.

Które operatory niosą jakość. Ablacja (Sekcja 4.4) wskazała, że jakość zależy głównie od operatorów dokonujących większych, strukturalnych zmian tras – niezbędne okazały się *invert* (wewnątrz jednej trasy) oraz *cross-route* (między trasami), po jednym z każdego typu – podczas gdy operatory zmieniające rozwiązanie drobniej (*swap*, *exchange*) są zbędne. Kierunek ten jest spójny z wcześniejszym wnioskiem o architekturze: na rozległej przestrzeni permutacyjnej rozważanego problemu liczy się zdolność do większych, strukturalnych zmian rozwiązania, a nie do drobnych korekt. Warto też odnotować, że waga nadana operatorowi przez strojenie nie jest miarą jego przydatności – operator *or-opt* mimo wysokiej wagi okazał się zbędny – co przypomina, że strojenie i ablacja odpowiadają na różne pytania.

Kontekst literaturowy. Wyniku tego nie należy uogólniać: pytanie, która architektura jest lepsza, nie ma w literaturze jednoznacznej odpowiedzi i zależy od problemu. Durillo i in. [9] wykazali, że steady-state wersje NSGA-II i SPEA2 przewyższają generacyjne na większości z 21 ciągłych problemów testowych (choć są 10–20 razy wolniejsze obliczeniowo). Przewaga steady-state nie ogranicza się przy tym do problemów ciągłych: Prins [26], w jednokryterialnym problemie marszrutyzacji pojazdów (VRP), również uzyskał lepsze wyniki inkrementalnym (steady-state) schematem niż generacyjnym. Choć jest to problem jednokryterialny, a więc analogia luźniejsza, przykład ten pokazuje, że przewaga jednej z architektur nie wynika wprost z podziału na problemy ciągłe i kombinatoryczne. Nasz wynik jest przypadkiem przeciwnym, w którym dla rozważanego problemu planowania tras UAV lepsza okazuje się architektura generacyjna. Potwierdza to, że o wyborze architektury decydują cechy konkretnego problemu i jego krajobrazu zastosowania, a nie sama klasa problemu.

Czy wynik nie jest artefaktem implementacji. Komponenty odpowiedzialne za porównywanie rozwiązań i obliczanie hypervolume są wspólne dla wszystkich czterech wariantów, które różni jedynie schemat zastępowania i kryterium selekcji. Obserwowana przewaga architektury jest zatem przypisywalna właśnie tym elementom projektu, a nie różnicy w implementacji współdzielonego silnika obliczeniowego.

Rozdział 5

Podsumowanie

Celem niniejszej pracy była analiza porównawcza i czynnikowa wielokryterialnych algorytmów ewolucyjnych w zastosowaniu do problemu planowania tras bezzałogowych statków powietrznych w misjach poszukiwawczo-ratowniczych. Problem sformułowano jako wariant problemu marszrutyzacji pojazdów z dwoma sprzecznymi kryteriami, oboma podlegającymi minimalizacji: czasem zakończenia misji oraz współczynnikiem nieodkrytych ofiar. Rdzeniem pracy jest plan czynnikowy 2×2 , w którym porównano cztery warianty algorytmu ewolucyjnego różniące się dwoma czynnikami projektowymi – architekturą (steady-state vs. generacyjna) oraz kryterium selekcji środowiskowej (wkład do hypervolume vs. crowding distance) – w celu ustalenia, który z tych czynników decyduje o jakości rozwiązań. Praca rozszerza wcześniejsze badania Trojanowskiego i in. [27, 28] o systematyczne, czynnikowe porównanie algorytmów oraz automatyczne strojenie ich hiperparametrów.

5.1. Realizacja celów pracy

W ramach pracy zaimplementowano cztery warianty algorytmu ewolucyjnego (SMS-EMOA, NSGA-II, SS-NSGA-II, GEN-SMS-EMOA), stanowiące cztery komórki planu czynnikowego, wraz z dedykowaną reprezentacją rozwiązania, czterema strategiami inicjalizacji populacji, operatorem krzyżowania porządkowego adaptowanym dla rozwiązań wielotrasowych oraz sześcioma operatorami mutacji. Przeprowadzono cztery eksperymenty obliczeniowe – inicjalizacyjny, czynnikowy, strojenie hiperparametrów oraz ablację operatorów mutacji – obejmujące łącznie 7 440 uruchomień weryfikacyjnych algorytmu¹ (nie licząc przebiegów wykonanych wewnątrz kampanii strojenia irace), na ośmiu instancjach testowych o zróżnicowanej charakterystyce.

5.2. Główne wyniki

Przeprowadzone badania pozwoliły sformułować cztery główne wnioski.

Inicjalizacja hybrydowa kluczowa dla wydajności. Eksperyment inicjalizacyjny wykazał, że hybrydowa inicjalizacja populacji, łącząca strategie heurystyczne z losowymi,

¹ Rozbicie (30 ziaren na każdą kombinację wariant–instancja): eksperyment inicjalizacyjny 480; plan czynnikowy 2×2 960 (4 warianty); niezależne strojenie hiperparametrów 960 (4 warianty); ablacja operatorów 5 040 (21 konfiguracji na ścieżce zachłannej $\times$ 8 instancji $\times$ 30 ziaren). Suma: 7 440.

daje znacznie lepsze wyniki niż inicjalizacja czysto losowa – wyższe HV na każdej z ośmiu instancji, przy całkowitej separacji próbek (wskaźnik Vargha-Delaneya $A = 1,000$). Różnica ta była największą obserwowaną w całej pracy, co podkreśla znaczenie odpowiedniego przygotowania populacji startowej dla problemów o dużej przestrzeni rozwiązań. Ponieważ inicjalizacja jest komponentem wspólnym dla wszystkich wariantów, przyjęto ją jako domyslną w pozostałych eksperymentach.

O jakości decyduje architektura, nie kryterium selekcji. Systematyczny plan czynnikowy 2×2 wykazał, że spośród dwóch czynników projektowych – architektury (steady-state vs. generacyjna) oraz kryterium selekcji (wkład do hypervolume vs. crowding distance) – mierzalny wpływ na jakość wyników ma wyłącznie architektura. Warianty generacyjne osiągnęły wyższe HV na siedmiu z ośmiu instancji, a efekt jest istotny statystycznie i odporny na usunięcie dowolnej pojedynczej instancji (analiza leave-one-out). Kryterium selekcji nie wykazało wpływu na żadnej z ośmiu instancji, a oba czynniki nie oddziałują na siebie, co pozwala interpretować je rozłącznie. Siła przewagi architektury zależy przy tym od trudności instancji: koncentruje się na najtrudniejszej instancji tcW (różnica median $\approx 0,16$ w skali HV), natomiast zanika tam, gdzie wszystkie warianty osiągają zbliżone, wysokie wyniki.

Wyniku tego nie należy uogólniać: w literaturze korzystniejsza architektura zależy od problemu – Durillo i in. [9] oraz Prins [26] obserwowali przewagę steady-state (odpowiednio na problemach ciągłych i w jednokryterialnej marszrutyzacji pojazdów), podczas gdy w rozważanym problemie lepsza okazuje się architektura generacyjna. Możliwym wyjaśnieniem jest lepsze utrzymanie różnorodności przez schemat generacyjny [15]. Mechanizm ten oraz pełny kontekst literaturowy omówiono w dyskusji wyników (Sekcja 4.5).

Strojenie poprawia wyniki, lecz nie odwraca hierarchii. Niezależne strojenie narzędziem irace poprawiło jakość każdego z czterech wariantów względem konfiguracji domyslniej, przy czym trzy z nich zyskały wyraźnie (duży efekt), a jeden – wariant steady-state z crowding distance (SS-NSGA-II) – najmniej. Strojenie nie odwróciło jednak hierarchii architektur: po dostrojeniu na czele pozostają dwa warianty generacyjne ($HV = 0,952$ i $0,949$), statystycznie nierozróżnialne między sobą, przed dwoma wariantami steady-state ($0,935$ i $0,912$). Dobór hiperparametrów ma zatem na rozważanym problemie znaczny wpływ na osiąganą jakość, lecz nie stanowi substytutu dla wyboru architektury. Nierozróżnialność dwóch czołowych wariantów, różniących się jedynie kryterium selekcji, ponownie potwierdza brak wpływu tego kryterium.

Trzy operatory mutacji wystarczają. Ablacja operatorów, przeprowadzona na najlepszym dostrojonym wariantcie, wykazała, że spośród sześciu operatorów mutacji dwa są niezbędne (*invert* i *cross-route* – po jednym operatorze wewnątrz- i między-trasowym), a trzy można usunąć bez pogorszenia jakości – najlepszy zestaw liczy zaledwie trzy operatory (*invert*, *cross-route*, *transfer*). Co istotne, operator *or-opt*, któremu strojenie irace nadało wysoką wagę, okazał się przy tym zbędny (jego usunięcie nieznacznie poprawia HV). Pokazuje to, że ablacja odpowiada na inne pytanie niż strojenie – które komponenty faktycznie się liczą – i jest wobec niego komplementarna.

5.3. Wkład pracy

Najważniejszym wkładem niniejszej pracy jest systematyczne, czynnikowe porównanie wielokryterialnych algorytmów ewolucyjnych w kontekście problemu optymalizacji tras UAV w misjach SAR – w dostępnej autorowi literaturze nie znaleziono analizy tego typu dla tego problemu. W szczególności praca:

- przeprowadziła plan czynnikowy 2×2 rozkładający konstrukcję algorytmu na dwa niezależne czynniki projektowe (architekturę oraz kryterium selekcji środowiskowej) i identyfikujący architekturę jako czynnik dominujący, a kryterium selekcji jako nieistotne,
- przeprowadziła *niezależne strojenie hiperparametrów dla każdego z czterech porównywanych algorytmów*, usuwając zarzut, że gorszy wynik danego wariantu wynika ze wspólnych, arbitralnie przyjętych hiperparametrów, i potwierdzając, że hierarchia architektoniczna nie jest artefaktem doboru parametrów,
- przeprowadziła ablację operatorów mutacji na najlepszym dostrojonym wariancie, wskazując, które operatory są niezbędne, a które zbędne, i pokazując, że analiza ta jest komplementarna do automatycznego strojenia,
- wprowadziła do rozważanego problemu, w którym we wcześniejszych badaniach stosowano algorytm SPEA2, rodzinę algorytmów opartych na sortowaniu niezdominowanym (SMS-EMOA, NSGA-II oraz ich warianty pośrednie).

5.4. Ograniczenia i kierunki dalszych badań

Jak każde badanie empiryczne, praca ma określony zakres. Kluczowe zagrożenia dla trafności wniosków zaadresowano już w jej ramach: niezależne strojenie każdego wariantu usunęło zarzut nierównego doboru hiperparametrów, analiza leave-one-out wykazała, że wynik nie opiera się na pojedynczej instancji, a analiza krzywych zbieżności potwierdziła wystarczalność przyjętego budżetu. Poniżej wskazano pozostałe granice zakresu badań oraz wynikające z nich naturalne kierunki dalszych prac.

- **Uwarunkowanie reżimem problemu.** Siła efektu architektury zależy od trudności instancji, a ocena przydatności operatorów mutacji – od tego, czy hiperparametry dostrożono pod konkretny ich zestaw. Wnioski należy odnosić do zbadanego reżimu problemu, a nie traktować jako uniwersalne dla całej klasy algorytmów ewolucyjnych.
- **Skończony budżet obliczeniowy.** Budżet 30 000 ewaluacji ustalono na podstawie stabilizacji krzywych zbieżności. Na najtrudniejszej instancji *tcW* krzywe *HV* nie osiągają wprowadzie pełnego nasycenia, jednak w końcowej fazie budżetu warianty generacyjne poprawiają się co najmniej tak szybko jak steady-state, więc dystans między architekturami nie wykazuje tendencji do zamykania się. Odwrócenie wyniku przy większym budżecie jest zatem mało prawdopodobne, choć jego bezpośrednie wykluczenie wymagałoby przebiegów o wydłużonym budżecie.
- **Zakres porównań.** We wcześniejszych badaniach nad tym problemem stosowano algorytm SPEA2 oraz przeszukiwanie lokalne [27]; niniejsza praca nie zawiera bez-

pośredniego porównania z tymi metodami, a wszystkie porównania prowadzone są wewnątrz rodziny czterech badanych wariantów.

- **Zakres instancji i liczby dronów.** Badania objęły osiem instancji testowych oraz ustaloną liczbę dronów. Rozszerzenie zbioru o scenariusze o większej skali oraz zbadanie wpływu liczby UAV pozwoliłoby pełniej ocenić skalowalność metody.
- **Generalizacja i odporność strojenia.** Strojenie jest częściowo obciążone dopasowaniem do instancji treningowych (poprawa mniejsza na zbiorze walidacyjnym niż treningowym). Ponadto strojenie optymalizujące zagregowaną jakość może pogarszać wynik na pojedynczych, trudnych instancjach: dostrojona konfiguracja SS-NSGA-II poprawiła siedem instancji, lecz regresowała na najtrudniejszej tcW. Naturalnym kierunkiem jest strojenie odporne (np. minimalizujące najgorszy przypadek lub prowadzone per instancja) oraz ocena generalizacji na większym zbiorze walidacyjnym.
- **Ujednolicenia względem wersji kanonicznych.** W celu izolacji badanych czynników selekcję rodziców ujednolicono do wyboru losowego, a wszystkie warianty wyposażono w ujednolicone zewnętrzne archiwum rozwiązań niezdominowanych. Badane warianty różnią się przez to od swoich wersji kanonicznych (które w przypadku NSGA-II stosują selekcję turniejową i nie korzystają z archiwum), więc wnioski o nich nie przenoszą się wprost na te wersje. Architektura generacyjna zachowała jednak przewagę mimo tych ujednoliczeń, co raczej wzmacnia niż osłabia wniosek o jej znaczeniu; naturalnym rozszerzeniem byłoby zbadanie selekcji rodziców jako trzeciego czynnika w planie $2 \times 2 \times 2$.
- **Strojenie parametrów inicjalizacji.** Proporcje czterech strategii inicjalizacji hybrydowej (Sekcja 2.3) dobrano wstępnie i nie były one przedmiotem strojenia ani analizy wrażliwości; eksperyment inicjalizacyjny (Sekcja 4.1) potwierdza przewagę inicjalizacji hybrydowej, lecz nie rozstrzyga o optymalności przyjętych proporcji.

Obiecującym kierunkiem dalszych badań jest również zastosowanie analizy krajobrazu przystosowania (*fitness landscape analysis*) do teoretycznego wyjaśnienia, dlaczego konkretne operatory mutacji (zwłaszcza *invert* i *cross-route*) okazały się szczególnie skuteczne dla rozważanego problemu, a także bezpośredni pomiar różnorodności populacji, który pozwoliłby zweryfikować hipotezę wyjaśniającą przewagę architektury generacyjnej.

Bibliografia

- [1] Andrea Arcuri and Lionel Briand. A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering. *Software Testing, Verification and Reliability*, 24(3):219–250, 2014.
- [2] Nicola Beume, Boris Naujoks, and Michael Emmerich. SMS-EMOA: Multiobjective selection based on dominated hypervolume. *European Journal of Operational Research*, 181(3):1653–1669, 2007.
- [3] Julian Blank and Kalyanmoy Deb. GECCO 2019 Bi-objective Traveling Thief Problem competition — evaluation protocol. <https://www.egr.msu.edu/coinlab/blankjul/gecco19-thief/>, 2019. Dane zostały pobrane w dniu 24.05.2026.
- [4] Tauã M. Cabreira, Lisane B. Brisolara, and Paulo R. Ferreira Jr. Survey on coverage path planning with unmanned aerial vehicles. *Drones*, 3(1):4, 2019.
- [5] Lawrence Davis. Applying adaptive algorithms to epistatic domains. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI 1985)*, pages 162–164, 1985.
- [6] Kalyanmoy Deb and Himanshu Jain. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints. *IEEE Transactions on Evolutionary Computation*, 18(4):577–601, 2014.
- [7] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.
- [8] Joaquín Derrac, Salvador García, Daniel Molina, and Francisco Herrera. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation*, 1(1):3–18, 2011.
- [9] Juan J. Durillo, Antonio J. Nebro, Francisco Luna, and Enrique Alba. On the effect of the steady-state selection scheme in multi-objective genetic algorithms. In *Evolutionary Multi-Criterion Optimization (EMO 2009)*, volume 5467 of *Lecture Notes in Computer Science*, pages 183–197. Springer, 2009.
- [10] A.E. Eiben and J.E. Smith. *Introduction to Evolutionary Computing*. Springer, 2003.
- [11] Michael T.M. Emmerich and André H. Deutz. A tutorial on multiobjective optimization: Fundamentals and evolutionary methods. *Natural Computing*, 17(3):585–609, 2018.
- [12] Chris Fawcett and Holger H. Hoos. Analysing differences between algorithm configurations through ablation. *Journal of Heuristics*, 22(3):431–458, 2016.
- [13] Marc Fleischer. The measure of Pareto Optima: Applications to multi-objective metaheuristics. In *Evolutionary Multi-Criterion Optimization (EMO 2003)*, volume 2632 of *Lecture Notes in Computer Science*, pages 519–533. Springer, 2003.

- [14] Enric Galceran and Marc Carreras. A survey on coverage path planning for robotics. *Robotics and Autonomous Systems*, 61(12):1258–1276, 2013.
- [15] David E. Goldberg and Kalyanmoy Deb. A comparative analysis of selection schemes used in genetic algorithms. In Gregory J. E. Rawlins, editor, *Foundations of Genetic Algorithms*, volume 1, pages 69–93. Morgan Kaufmann, 1991.
- [16] Myles Hollander, Douglas A. Wolfe, and Eric Chicken. *Nonparametric Statistical Methods*. John Wiley & Sons, 3 edition, 2013.
- [17] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [18] Hisao Ishibuchi, Ryo Imada, Yu Setoguchi, and Yusuke Nojima. Reference point specification in hypervolume calculation for fair performance comparison and efficient search. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2017)*, pages 585–592, 2017.
- [19] Jan K. Lenstra and Alexander H.G. Rinnooy Kan. Complexity of vehicle routing and scheduling problems. *Networks*, 11(2):221–227, 1981.
- [20] Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Mauro Birattari, and Thomas Stützle. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016.
- [21] Mingyang Lyu, Yibo Zhao, Chao Huang, and Hailong Huang. Unmanned aerial vehicles for search and rescue: A survey. *Remote Sensing*, 15(13):3266, 2023.
- [22] Douglas C. Montgomery. *Design and Analysis of Experiments*. John Wiley & Sons, Hoboken, NJ, 9 edition, 2017.
- [23] Ilhan Or. *Traveling Salesman-Type Combinatorial Problems and Their Relation to the Logistics of Regional Blood Banking*. PhD thesis, Northwestern University, 1976.
- [24] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 5 edition, 2016.
- [25] Jean-Yves Potvin and Jean-Marc Rousseau. An exchange heuristic for routing problems with time windows. *Journal of the Operational Research Society*, 46(12):1433–1446, 1995.
- [26] Christian Prins. A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research*, 31(12):1985–2002, 2004.
- [27] Krzysztof Trojanowski, Artur Mikitiuk, Jakub Grzeszczak, and Frédéric Guinand. Complete coverage and path planning for emergency response by UAVs in disaster areas. In *Computational Collective Intelligence (ICCCI 2023)*, volume 14162 of *Lecture Notes in Artificial Intelligence*, pages 647–659. Springer, 2023.
- [28] Krzysztof Trojanowski, Artur Mikitiuk, Jakub Grzeszczak, and Frédéric Guinand. Multiobjective optimization of complete coverage and path planning for emergency response by UAVs in disaster areas. In *Computational Science (ICCS 2024)*, volume 14836 of *Lecture Notes in Computer Science*, pages 150–165. Springer, 2024.
- [29] András Vargha and Harold D. Delaney. A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics*, 25(2):101–132, 2000.
- [30] Qingfu Zhang and Hui Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6):712–731, 2007.
- [31] Eckart Zitzler, Marco Laumanns, and Lothar Thiele. SPEA2: Improving the strength Pareto evolutionary algorithm. TIK-Report 103, ETH Zurich, Computer Engineering and Networks Laboratory (TIK), 2001.

- [32] Eckart Zitzler and Lothar Thiele. Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation*, 3(4):257–271, 1999.
- [33] Eckart Zitzler, Lothar Thiele, Marco Laumanns, Carlos M. Fonseca, and Viviane G. da Fonseca. Performance assessment of multiobjective optimizers: An analysis and review. *IEEE Transactions on Evolutionary Computation*, 7(2):117–132, 2003.

Dodatek A — opis kodu

Pełny kod źródłowy implementacji wraz ze skryptami eksperymentów i analizy statystycznej dostępny jest publicznie w repozytorium GitHub². Poniżej przedstawiono organizację repozytorium oraz sposób uruchamiania eksperymentów.

Struktura repozytorium

Kod zorganizowano w następujące katalogi i moduły:

<code>main.py</code>	interfejs wiersza poleceń (pojedyncze uruchomienia)
<code>src/algorithm/</code>	cztery warianty algorytmu, operatory, inicjalizacja
<code>src/model/</code>	reprezentacja rozwiązania i funkcje celu
<code>src/io/</code>	wczytywanie instancji testowych z plików JSON
<code>experiments/</code>	skrypty eksperymentów batchowych
<code>analysis/</code>	analiza statystyczna (Wilcoxon, Holm, Vargha-Delaney)
<code>tuning/</code>	konfiguracja i skrypty narzędzia irace
<code>data/</code>	pliki JSON definiujące instancje testowe
<code>tests/</code>	testy jednostkowe operatorów i funkcji celu
<code>results/</code>	surowe wyniki przebiegów oraz raporty analiz

Katalog `results/` zawiera także wyniki dodatkowych sprawdzeń wykonanych w toku prac (m.in. walidację poprawności implementacji oraz kontrolę wrażliwości na definicję okna oceny T_{cut}), niewłączonych do zasadniczej części pracy.

Uruchamianie eksperymentów

Pojedyncze uruchomienie algorytmu odbywa się przez interfejs wiersza poleceń z parametrami określającymi wariant algorytmu, instancję testową, ziarno generatora liczb pseudolosowych oraz wartości hiperparametrów. Przykładowe wywołanie uruchamiające algorytm SMS-EMOA na instancji `tcB` z ustalonym ziarnem:

```
python main.py --algorithm sms_emoa --instance tcB \
               --seed 42 --population 100 --max-eval 30000
```

Wynik uruchomienia zapisywany jest w formacie JSON i zawiera konfigurację uruchomienia, historię przebiegu (wartości HV , f_1 , f_2 w kolejnych generacjach) oraz archiwum końcowe rozwiązań niezdominowanych. Eksperymenty batchowe obejmujące wiele kombinacji *algorytm* $\times$ *instancja* $\times$ *ziarno* realizowane są przez skrypty w katalogu

² <https://github.com/adak101/sms-emoa-uav-path-planning>

`experiments/`, wykorzystujące mechanizm `ProcessPoolExecutor` do równoległego wykonania.

Każde uruchomienie jest deterministyczne dla danego ziarna generatora liczb pseudolosowych, co zapewnia pełną powtarzalność wyników eksperymentów.